

Manual Técnico para el desarrollo de un Robot Diferencial como plataforma de Desarrollo en Robótica Móvil.

ELABORADO POR EL
EQUIPO DE SERVICIO SOCIAL
ROBOKIT LAVIRIA

CONFORMADO POR:

Baeza Collazo Yahir Alejandro
Barrón Martínez Jeshua Lestat
Gutiérrez Robledo Juan Armando





PROYECTO DE SERVICIO SOCIAL SEGUNDO COMPONENTE

DEMOSTRACIONES DE ROBÓTICA MÓVIL PARA LA DIFUSIÓN Y APROPIACIÓN SOCUAL DEL CONOCIMIENTO

Encargado: Ramírez Paredes Juan Pablo Ignacio

Miembros y contribuidores

Baeza Collazo Yahir Alejandro -Ing. Mecatrónica
Barrón Martínez Jeshua Lestat-Ing. Mecatrónica
Gutiérrez Robledo Juan Armando-Ing. Mecatrónica





CONTENIDO

1. INTRODUCCIÓN	1
2. PROPÓSITO DEL MANUAL.....	1
3. DESCRIPCIÓN TÉCNICA.....	2
4. ESTRUCTURA MECÁNICA	3
4.1 Diseño	3
4.2 Ensamble	9
5. CONTROL Y COMUNICACIONES	17
5.1 Arquitectura de control	17
5.2 Raspberry Pi	18
5.3 Arduino	18
6. HARDWARE.....	19
6.1 Alimentación	19
6.2 Actuadores	21
6.3 Placa PCB	22
6.4 Flujo de conexiones.....	26
7. SOFTWARE27	
7.1 Integración e interacción	28
7.2 Arduino	28
7.3 ROS	29
8. CONEXIÓN A LA RASPBERRY	31
8.1 Configuración de la Raspberry Pi	31
8.2 Configuración de Windows	33
8.3 Obtención la dirección IP de la Raspberry Pi	33
9. RANGE SENSING	33
9.1 Sensores.....	33
9.2 Disposición de sensores	34
9.3 Codificación Arduino	36
9.4 Espacio de trabajo (Workspace) para lectura de sensores.....	36



1. INTRODUCCIÓN

Los robots diferenciales se caracterizan por su diseño funcional y versatilidad de aplicaciones, constan de dos ruedas ubicadas en los costados de su estructura como una de sus características principales, lo que les permite llevar a cabo un control de movimiento preciso en función de la capacidad de tracción de cada rueda de forma independiente.

Su modelo cinemático ha sido ampliamente estudiado para obtener las ecuaciones de movimiento que rigen la interacción de un robot diferencial en base a sus especificaciones básicas de diseño, lo que los convierte en una primera alternativa ideal para todo aquel que desee adentrarse en el mundo de la robótica; pues la escalabilidad de estos prototipos no solo se reduce a la *odometría* o a la navegación autónoma por medio de un control sofisticado, sino que por medio de la fusión de múltiples sensores, actuadores y demás componentes, la estructura de un robot diferencial puede ser utilizada como el punto de partida ideal para desarrollar aplicaciones más sofisticadas, como el mapeo y recreación de entornos, la detección de objetos, la asistencia virtual o sistemas de vigilancia.

El presente manual abarca los principios básicos para la construcción de esa primera estructura funcional de un robot diferencial, teniendo como punto de interés el aprovechamiento y desarrollo del prototipo desde la perspectiva de algunos campos relacionados con el desarrollo de éste, como lo pueden ser la robótica móvil, visión por computadora, sistemas embebidos, inteligencia artificial, entre otros.

2. PROPÓSITO DEL MANUAL

El seguimiento de cada una de las etapas concernientes al desarrollo del prototipo, aseguran que, disponiendo de los elementos adecuados y las herramientas necesarias, cualquier persona con un mínimo interés en materia de robótica o afines, sea capaz de replicar un modelo funcional de un robot diferencial. Por tal motivo, la estructura de los contenidos del manual sugiere un orden bajo el cual el usuario complete y comprenda los requerimientos de una fase antes de incurrir en una nueva sección; de esta forma se evitan los errores presentes durante el desarrollo y se jerarquizan los procesos para evitar una falta de cohesión entre cada una de las etapas.

Sin embargo, es bien sabido que el desarrollo de un prototipo como el aquí presentado involucra múltiples áreas tales como el diseño asistido por computadora, la electrónica, programación, sistemas control, etcétera; por lo cual, si no se desea llevar el orden propuesto de forma secuencial por la numeración de los capítulos, el usuario es libre de

abordar la implementación del manual de forma distinta, siempre y cuando se refiera exclusivamente al apartado de su interés.

3. DESCRIPCIÓN TÉCNICA

El prototipo presentado es un robot diferencial compacto, de estructura ligera y fácil adaptabilidad, capaz de ser configurado como una herramienta de aprendizaje o para explorar el desarrollo personalizado de aplicaciones de navegación autónoma en espacios reducidos como primera instancia, puesto que la plataforma está preparada para la integración de múltiples componentes (no considerados en este manual).

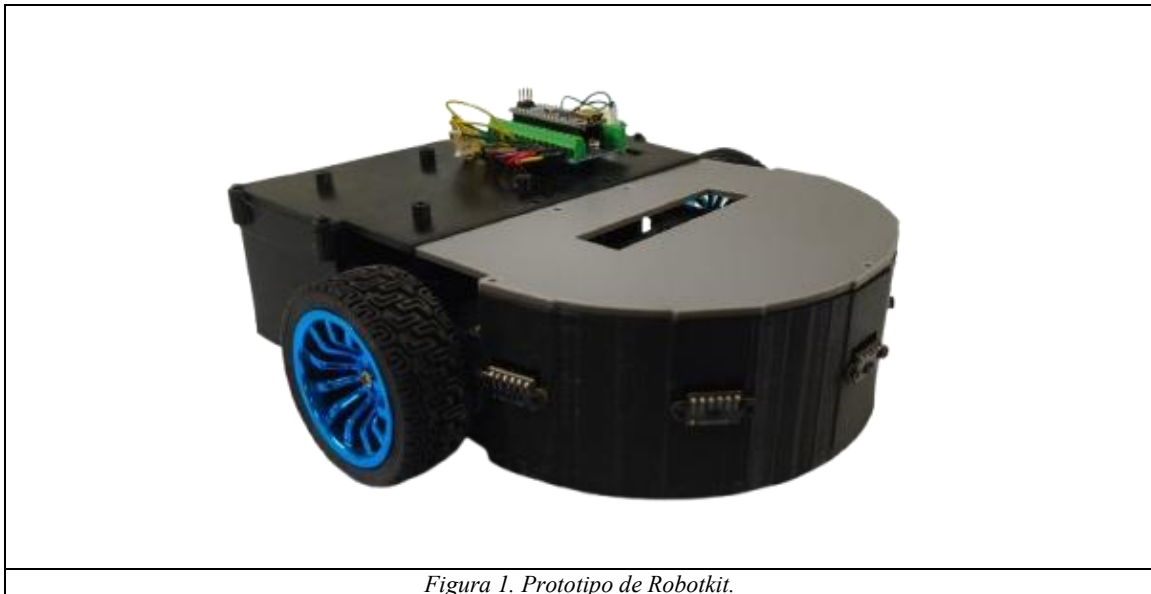
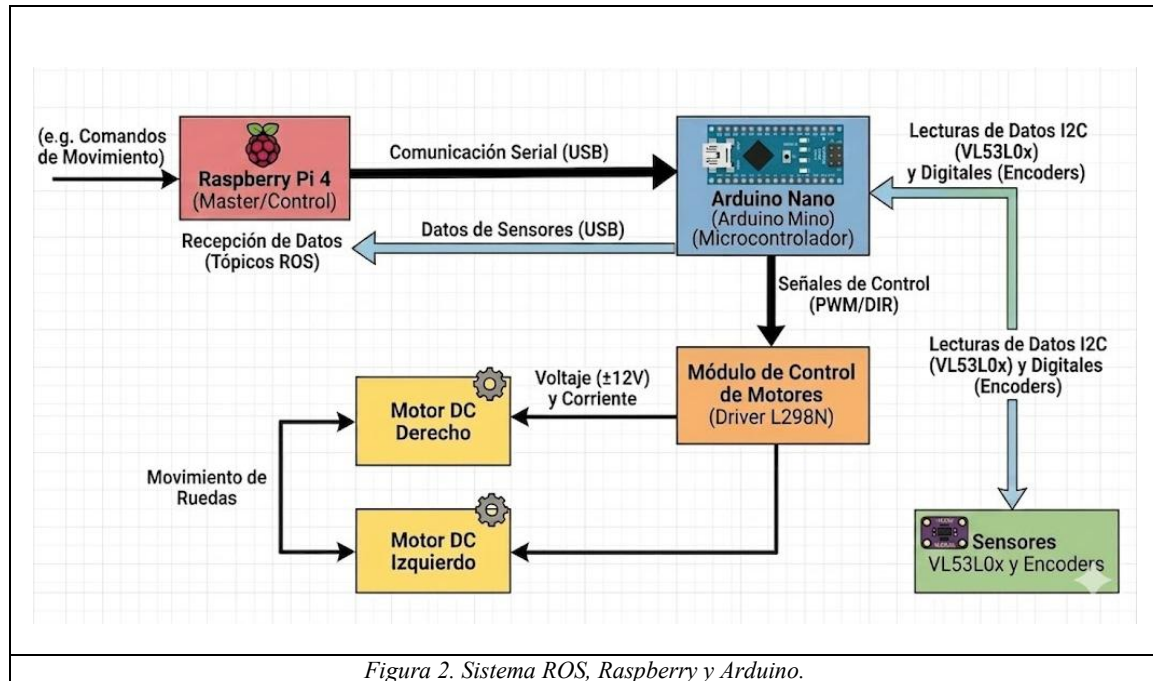


Figura 1. Prototipo de Robotkit.

Al tratarse inadvertidamente de un sistema de baja potencia (sin superar los *20 Voltios* y *3 Amperes* en ocasión alguna) el diseño y la adaptación del hardware satisfacen las condiciones para desempeñar el movimiento a través de dos ruedas motrices, alcanzando una velocidad máxima de *120 rpm* por lapsos prolongados de tiempo.

Refiriéndonos a la etapa de control, este prototipo ofrece una arquitectura abierta al desarrollo al involucrar una tarjeta Raspberry Pi como la principal unidad de procesamiento en el robot, trabajando en conjunto con un microcontrolador Arduino a través de nodos que gestionan la transferencia de instrucciones por medio de *ROS (Robot Operating System)*.

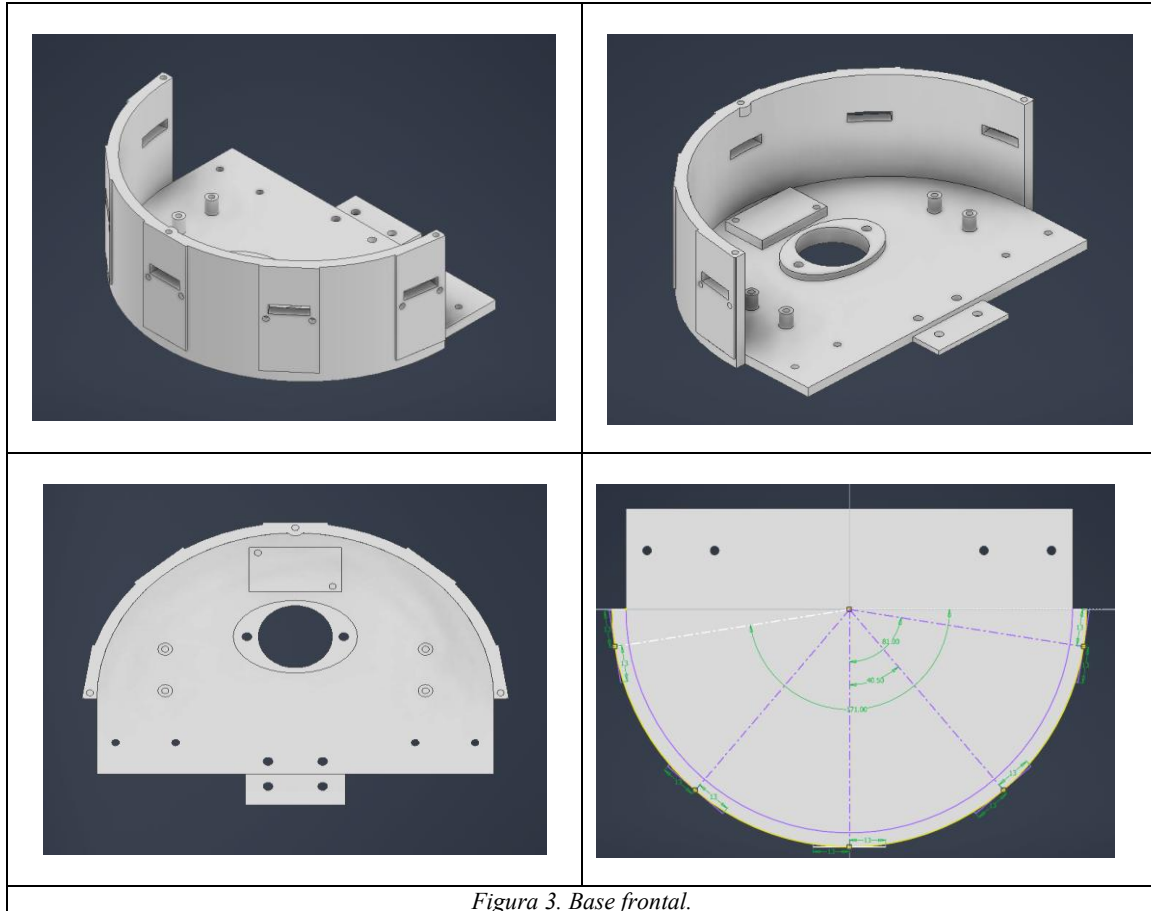


Características como las anteriormente descritas, le otorgan la tan deseada capacidad remota de control al robot diferencial, ofreciendo una experiencia interactiva al usuario al gestionar a través de máquinas virtuales y escritorios remotos, el flujo de trabajo ideal para el desarrollo, integrando inicialmente la disposición estratégica de sensores de distancia láser para la simulación de escenarios por medio de herramientas gráficas del propio ROS como *RViz* (*ROS Visualization*) o *Gazebo*, ambos simuladores de código abierto con la capacidad de probar algoritmos en entornos virtuales, de realizar depuraciones a través de la visualización en tiempo real del estado del robot, lectura de sensores, entre otros.

4. ESTRUCTURA MECÁNICA

4.1 Diseño

El diseño del chasis se enfocó en optimizar peso y espacio además de obtener una alta funcionabilidad. El chasis se integra por 8 piezas, 2 de estas constituyen la base principal, donde se incorporan componentes de sensado y potencia: sensores de proximidad VL5310X, reguladores LM2596, bases para motores (poner modelo), motores, modulo L298N, placa PCB, batería. La estructura superior del chasis se compone por 2 piezas las cuales están ensambladas sobre la base principal, estas incorporan el procesamiento principal con el Arduino Nano, la Raspberry Pi junto con un regulador de voltaje 7805. El ensamblaje de los componentes principales se logra mediante conectores auxiliares diseñados a la medida, asegurando una unión sólida entre las piezas.



La Figura 3 ilustra la sección frontal de la base principal del chasis, el diseño de su geometría se determinó a partir de la distribución de 5 sensores de distancia, los cuales se posicionaron con un ángulo de separación de 40.5° . Esta configuración se seleccionó considerando que el ángulo de visión (FOV) máximo del sensor es de 35° , evitando así que los haces de luz infrarroja emitidos interfieran con los receptores adyacentes y generen lecturas erróneas.

El contorno de la base integra alojamientos rectangulares y orificios diseñados para montar los sensores e integrar su cableado hacia la parte interna del chasis. Para mejorar la estabilidad y el equilibrio del sistema, se incorporó una base elipsoidal con orificios que atraviesan completamente el espesor de la base destinados para ensamblar una rueda loca. Asimismo, la estructura incluye 4 soportes cilíndricos para la fijación de la placa PCB de conexiones optimizando la gestión de cableado de los sensores (de los cuales se utilizan cuatro de sus cinco terminales).

En cuanto al ensamblaje mecánico, los cuatro orificios laterales permuten la unión con la base posterior través de las bases de los motores, mientras que los dos orificios de la parte intermedia aseguran la estructura mediante una pieza de acoplamiento auxiliar. Se integró una pequeña base rectangular para incluir un regulador de voltaje con el cual se alimentará

la placa PCB de los sensores. Finalmente, en la parte superior del contorno se añadieron 3 orificios con el objetivo de ensamblar la parte superior del chasis.

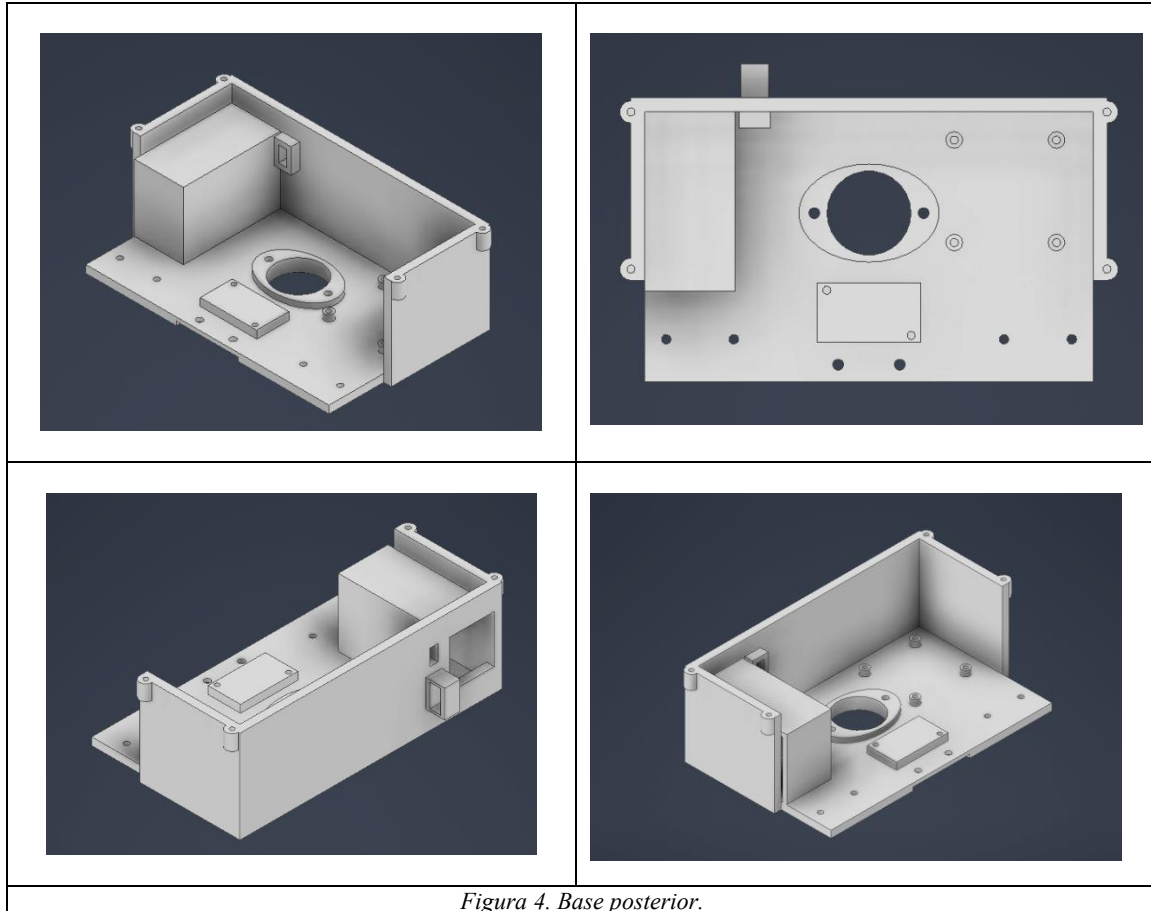


Figura 4. Base posterior.

La Figura 4, muestra la sección posterior de la base principal. Al igual que la sección frontal, esta integra una base elipsoidal con orificios pasantes diseñados para montar una segunda rueda loca, optimizando la estabilidad estática y dinámica del robot. Se distribuyeron cuatro soportes cilíndricos en un extremo para la fijación del módulo L298N, mientras que el extremo opuesto se diseñó un compartimiento para la batería de manera segura, esto permite un montaje y desmontaje simplificado, asimismo en el contorno trasero de la base se optó por incluir dos ranuras para incrustar los cables de la batería y evitar que estén sueltos. La base del regulador se colocó en la sección central, aprovechando la proximidad tanto a la batería como el controlador L298N para minimizar las caídas de tensión y optimizar la distribución de energía.

En cuanto al ensamblaje, los cuatro orificios laterales tienen el objetivo de ensamblar la base posterior con la base frontal por medio del soporte de los motores, de igual manera los orificios intermedios se encargan de ensamblar por medio de un acoplamiento auxiliar. Adicionalmente se incorporó una ranura de encastre bajo de los orificios intermedios para

incrementar la rigidez estructural del conjunto. Finalmente, en las cuatro esquinas del contorno posterior de esta sección se ubican cuatro pequeños soportes cilíndricos con el propósito de ensamblar la parte superior del chasis.

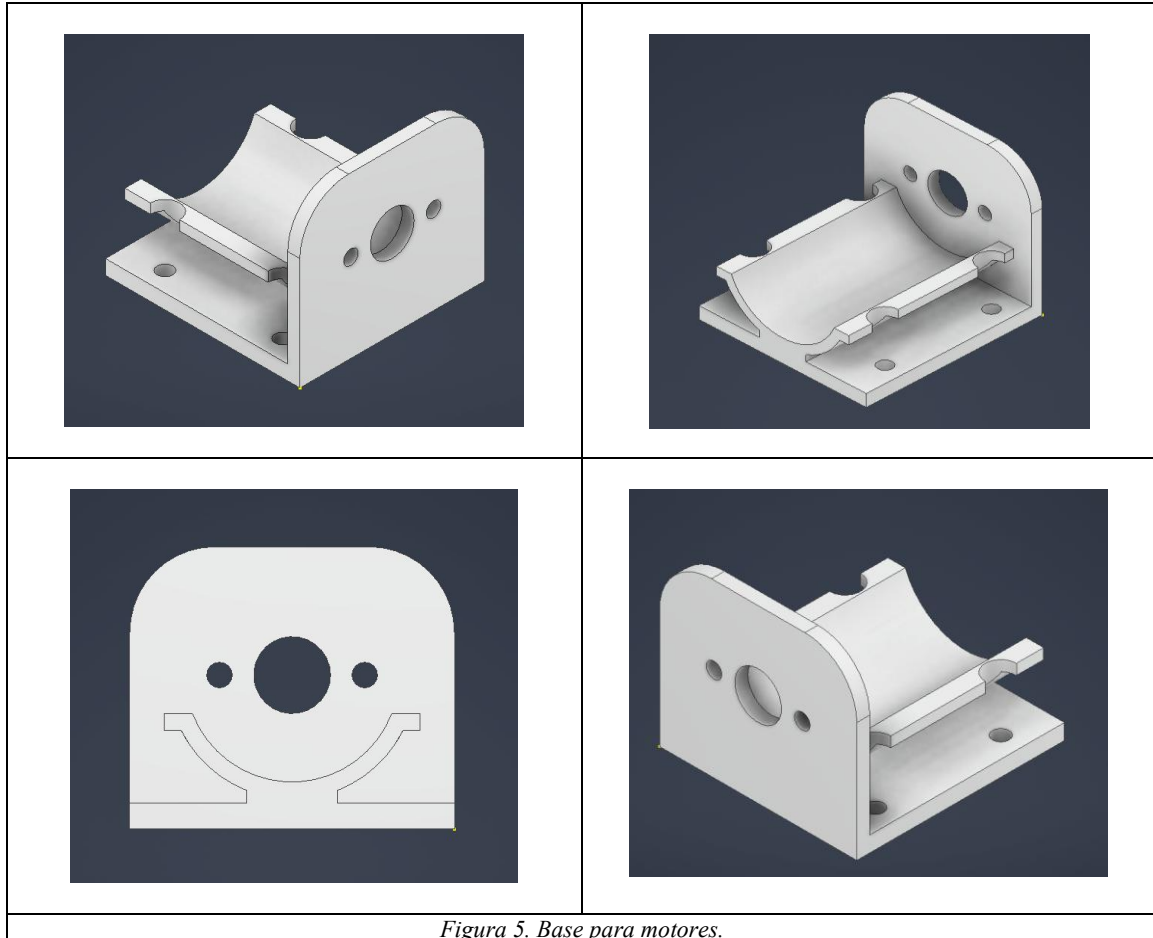
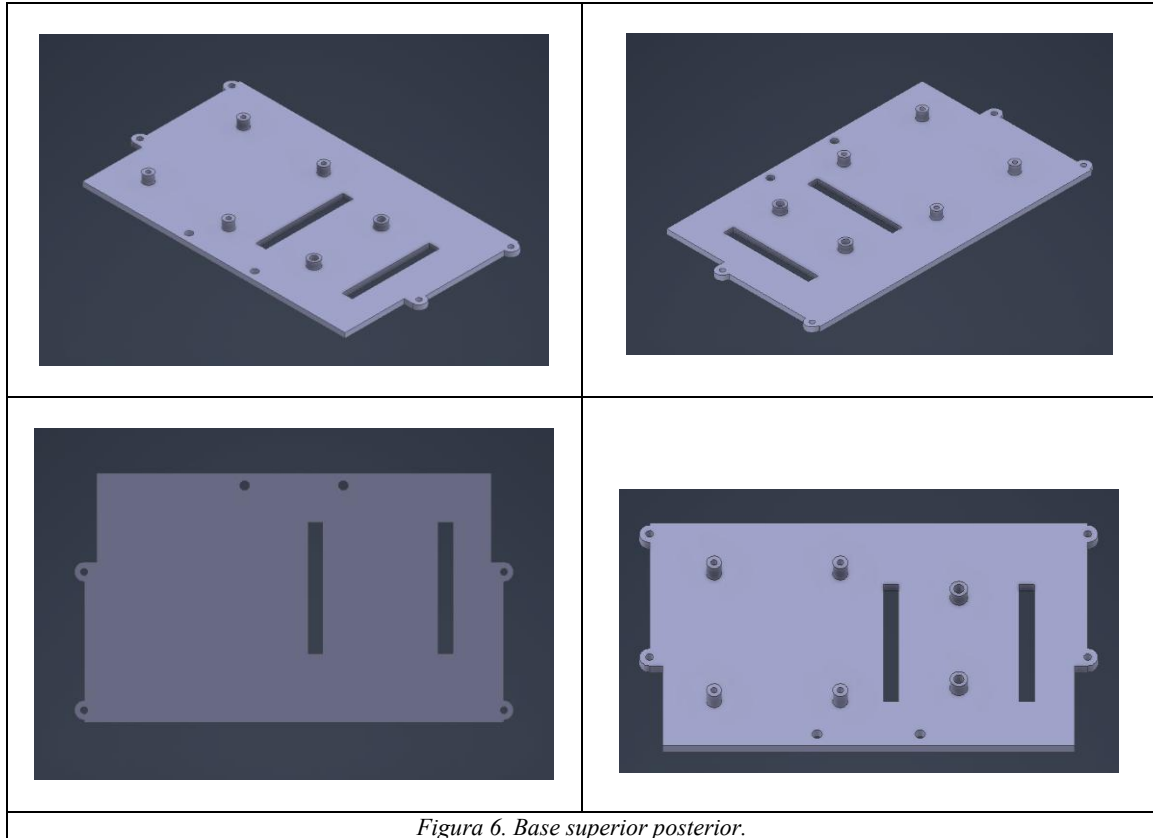


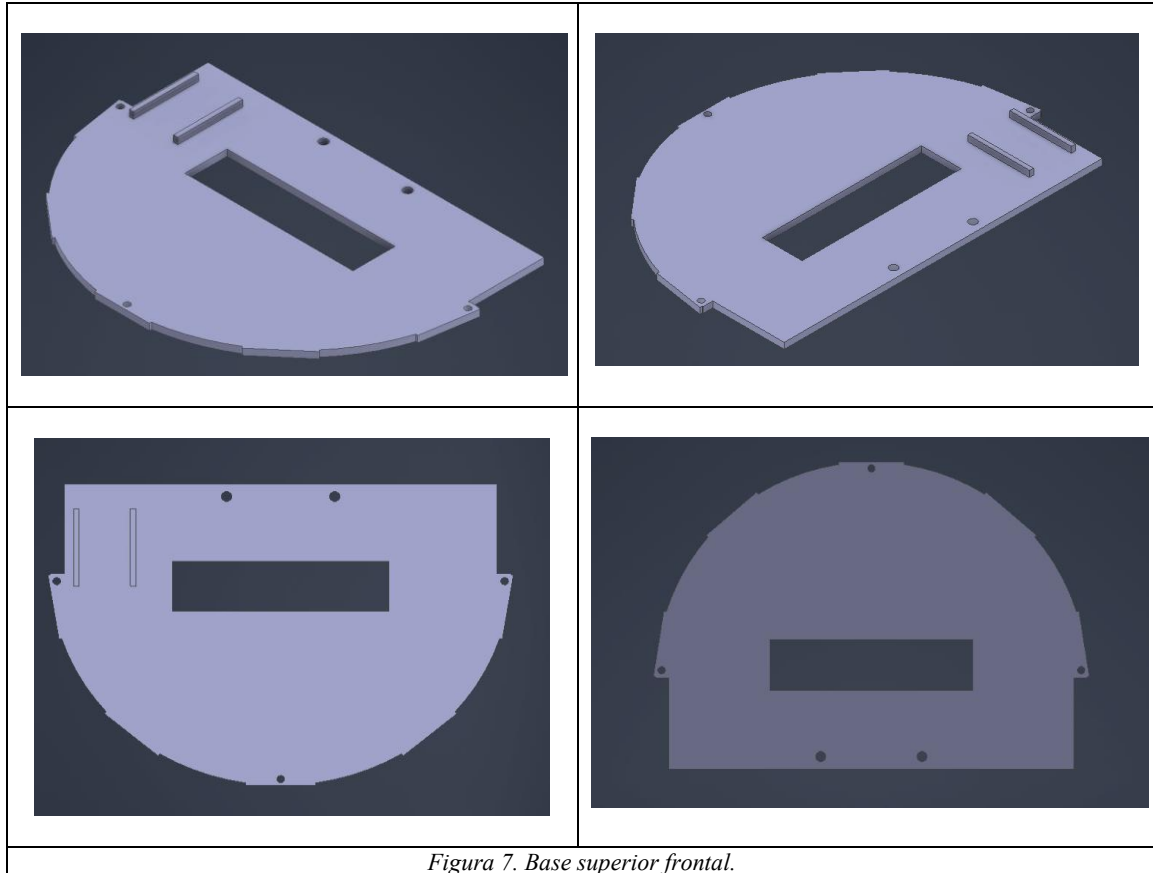
Figura 5. Base para motores.

La Figura 5, muestra el diseño personalizado de la base para los motores. Esta pieza se desarrolló para sustituir la base comercial incluida con los motores, la cual presentaba inestabilidad e incompatibilidad con la geometría del chasis. El nuevo diseño integra un alojamiento envolvente que sostiene la mayor parte del motor, proporcionando una mayor rigidez mecánica.

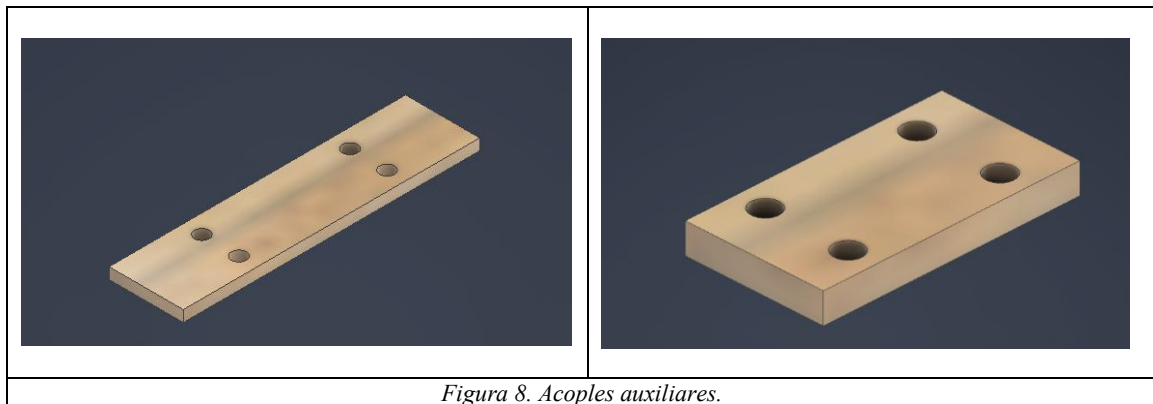
Para el ensamblaje, la pieza cuenta con cuatro orificios superiores concéntricos a cuatro perforaciones inferiores; esta disposición permite el paso de los tornillos que aseguran la unión estructural entre las secciones frontal y posterior de la base principal. En el plano frontal del soporte, se incluyeron dos orificios de fijación para el motor y un barreno de mayor diámetro diseñado para la alineación del eje de transmisión.



La Figura 6, muestra la sección superior posterior del chasis, diseñada como plataforma de control central. En uno de sus laterales, se integraron cuatro soportes cilíndricos para el montaje de la Raspberry Pi, mientras que en el extremo opuesto se colocaron dos soportes destinados a la base de conexiones del Arduino Nano. Con el fin de optimizar el cableado extenso y evitar interferencias, se incluyeron dos ranuras laterales que permiten gestionar mejor el cableado desde el compartimento interno hacia el microcontrolador. Finalmente, la pieza cuenta con cuatro extensiones cilíndricas en sus esquinas, las cuales actúan como puntos de anclaje para el ensamblaje estructural con la base principal del robot, además de contener dos orificios que permiten ensamblar con la sección superior frontal con un acople auxiliar.



La Figura 7, muestra la sección superior frontal del chasis. Esta pieza integra una ranura rectangular diseñada para el cableado de control desde los sensores hacia el Arduino Nano, facilitando además la interconexión de la batería con el regulador 7805. El regulador se ubicará en una cavidad con geometría de semi rectángulo, diseñada específicamente para asegurar su posición. Para el ensamblaje mecánico, se dispusieron tres orificios perimetrales que permiten el ensamblaje con la base principal; asimismo, los orificios intermedios funcionan como puntos de unión para acoplar ambas secciones de la parte superior mediante un soporte auxiliar.





La Figura 8, ilustra los acoples auxiliares implementados para el ensamblaje de las piezas principales. En el extremo izquierdo se observa el acople diseñado para la unión de las secciones superiores, mientras que en el extremo derecho se presenta el acople para el ensamblaje de las secciones inferiores o principales del chasis. Ambas piezas se fabricaron con un espesor de 3 mm , dimensionadas estratégicamente para evitar interferencias mecánicas con el montaje de motores o reguladores. Asimismo, este espesor permite la compatibilidad con tornillería comercial estándar, facilitando el mantenimiento y la solidez del chasis.

Las tolerancias para la creación de los orificios fueron de $\pm 0.01\text{ mm}$ debido a la calidad de la impresión y la localización de ciertos orificios, ya que el manipulador de la impresora puede llegar a crear una geometría con cierto porcentaje de error. En la ranura que se encarga de ensamblar las piezas principales de la base y el compartimiento de la batería se utilizaron tolerancias de $\pm 0.1\text{ mm}$, ya que normalmente las impresoras 3D requieren soportes al momento de imprimir una pieza, estos soportes dejan bordes pequeños en las piezas generando un mal ensamblaje.

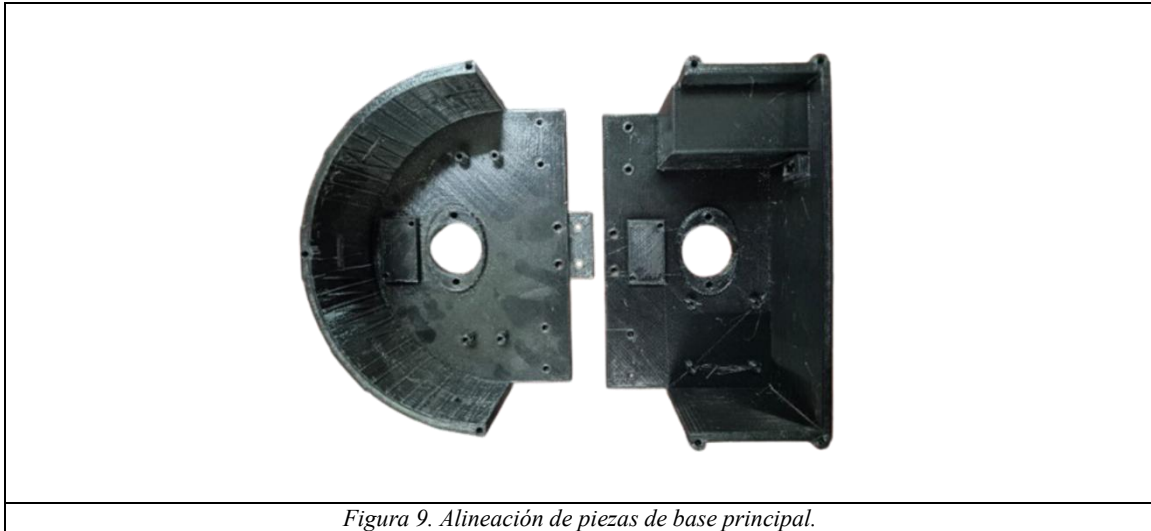
Las tolerancias aplicadas en los orificios se fijaron en $\pm 0.01\text{ mm}$, considerando la precisión dimensional de la impresora y la ubicación de los barrenos críticos. Esto se debe a que el sistema de extrusión y el movimiento de los ejes llegan a presentar ligeras desviaciones geométricas respecto al modelo CAD. Por otro lado, en la ranura de ensamblaje de la base principal y en el compartimiento de la batería, se emplearon tolerancias de $\pm 0.1\text{ mm}$. Este margen es necesario debido a que las estructuras de soporte generadas durante la impresión suelen dejar restos de material, rugosidades o bordes en la superficie; dichas imperfecciones llegan a dificultar el ensamblaje mecánico de las piezas.

Los archivos de las piezas se encuentran en el siguiente repositorio de GitHub:

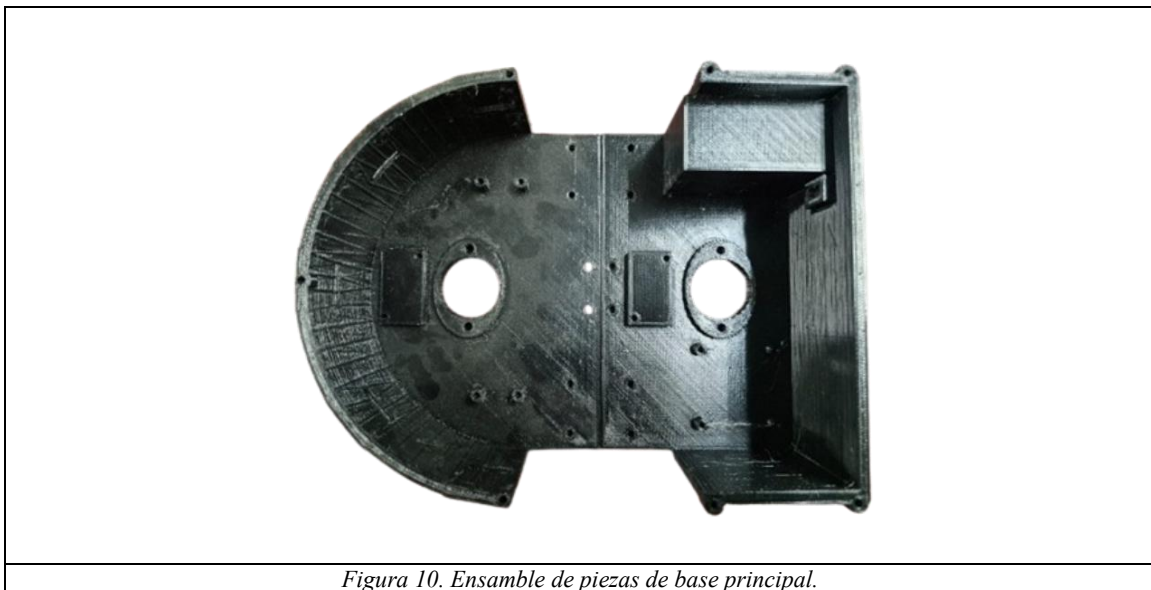
https://github.com/Yahir-Baeza-17/RoboKit_sensors.git

4.2 Ensamble

La secuencia de armado comienza con las dos piezas que conforman la base principal, para ensamblarlas se necesitan alinear de manera correcta como se ilustra en la Figura 9.



El siguiente paso será ensamblar a ambas piezas, asegurando que los 4 orificios que se encuentran en la parte central queden concéntricos, en la Figura 10 se muestra el ensamble.



Para sujetar las 2 piezas principales se utilizará el conector rectangular más pequeño de todas las piezas, para ello se deben de alinear los centros de los 4 orificios que contiene el conector auxiliar como se muestra en la Figura 11.

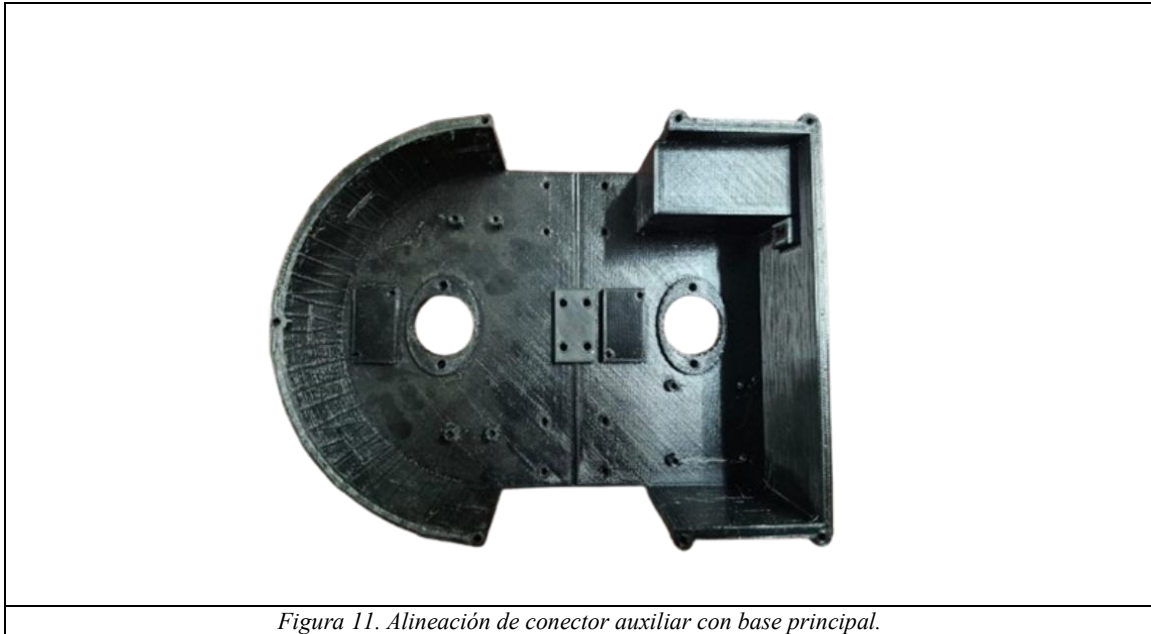


Figura 11. Alineación de conector auxiliar con base principal.

Una vez alineadas las piezas, por medio de 4 tornillos M4X16 y sus respectivas tuercas, se atornillaron y sujetaron las piezas para conformar una estructura rígida, en la Figura 12 se observan los resultados del ensamble.

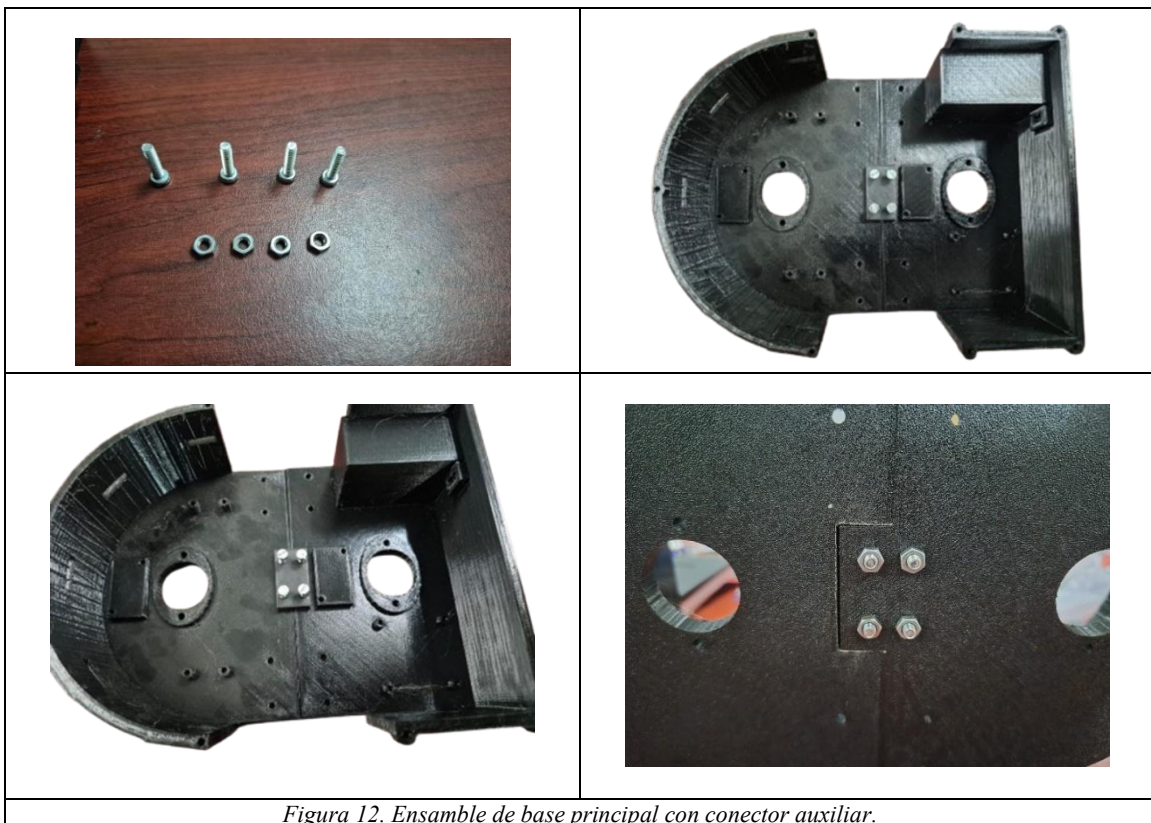


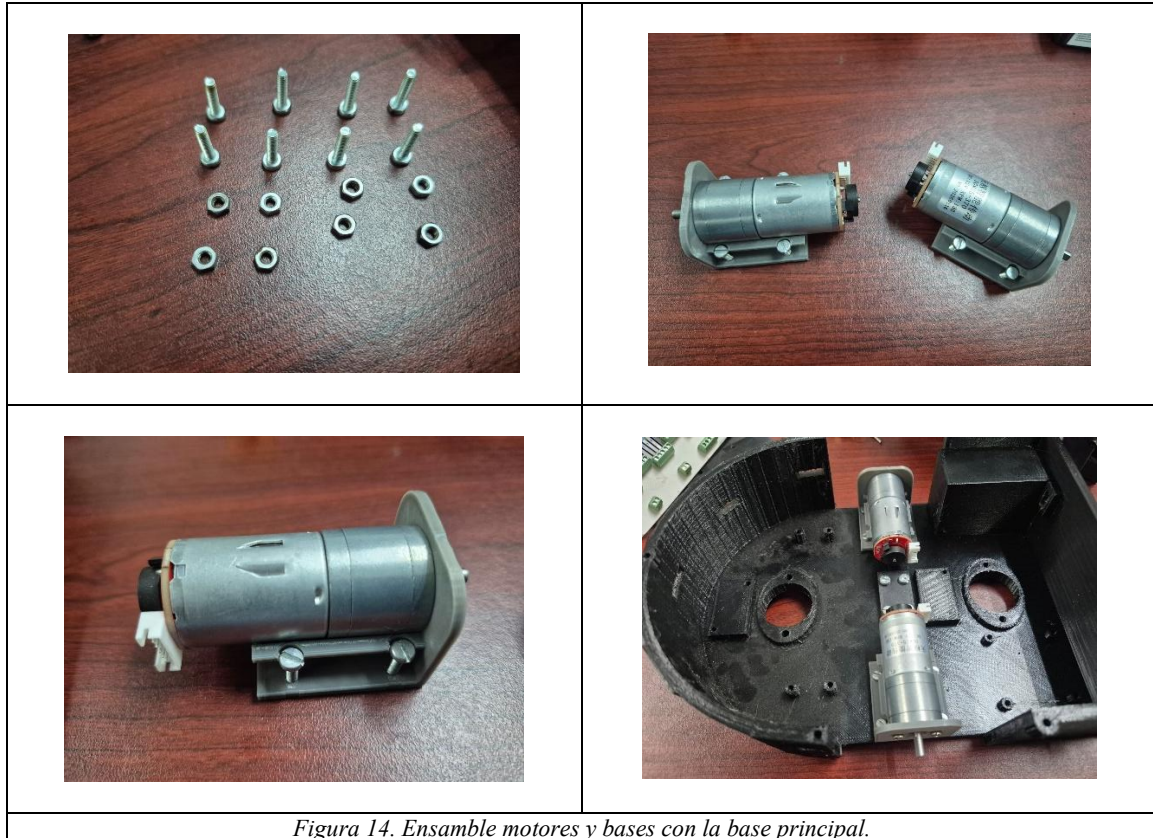
Figura 12. Ensamble de base principal con conector auxiliar.

El siguiente paso será ensamblar los motores con sus respectivas bases, para ello se utilizaron 4 tornillos M3X5, cada motor contiene 2 orificios en la parte frontal, las bases también contienen 2 orificios y un tercero por donde pasa el eje del motor en la parte frontal, para asegurar un buen ensamblaje lo mejor es alinear cada orificio de manera correcta como se ilustra en la Figura 13.

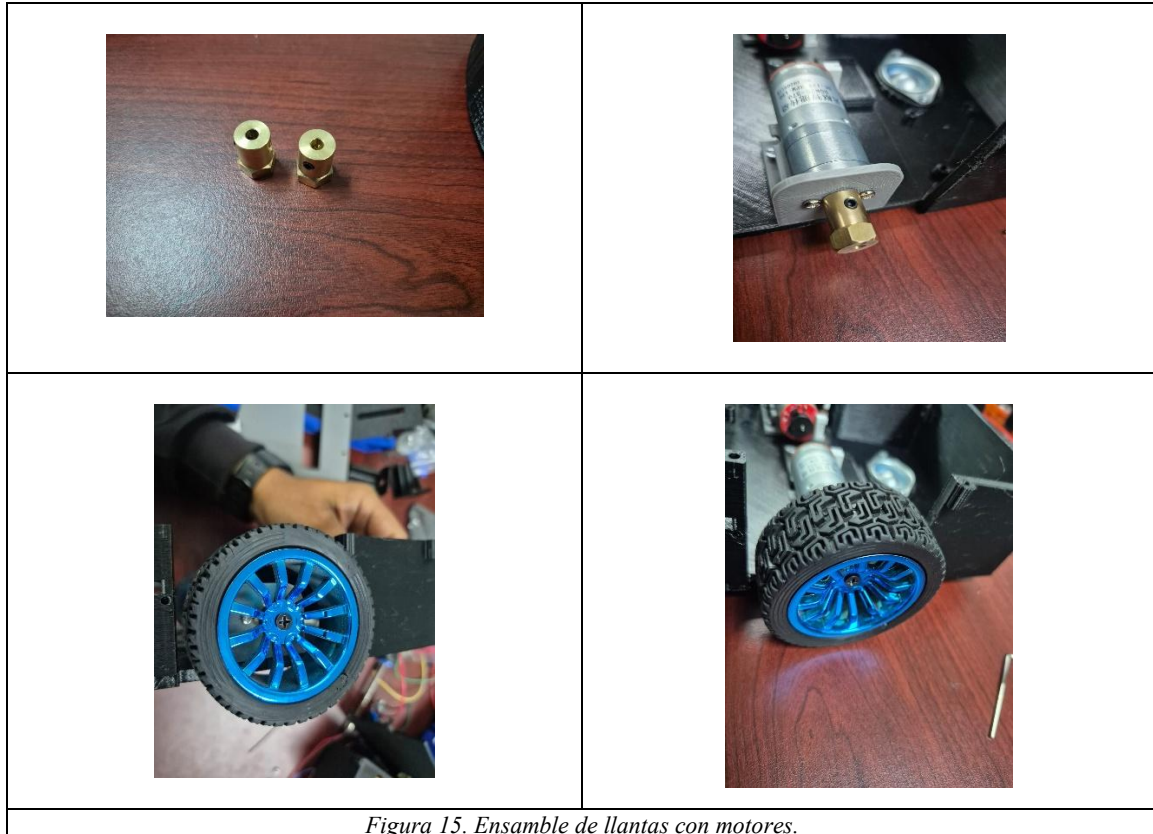


Figura 13. Ensamble de motores con sus respectivas bases.

Los motores junto con sus bases se colocarán a los costados de la base principal, alineados con los 4 orificios que se encuentran ahí, para ello se utilizaron 8 tornillos M3X12 con sus respectivas tuercas, cada tornillo se colocó en un orificio inferior de la base, después se alinearon a los 4 orificios laterales que contiene la base principal y por medio de las tuercas se ajustaron cada una de las bases, el ejemplo está en la Figura 14.



El ensamblaje de las ruedas de los motores se realizó con las piezas que ya tenía el kit de cada motor, en el eje de cada motor se puso la base metálica de cada rueda y se atornillo, después se montó la llanta y se atornillo de la parte frontal.



Para complementar el equilibrio en la base, se ensamblará una rueda loca en la parte frontal y otra en la posterior, en el proceso se utilizaron 4 tornillos M3X16 con sus respectivas tuercas. Para este ensamble solo se alinearon los orificios de la rueda loca con los orificios de la base helicoidal como se muestra en la Figura 16.



Una vez teniendo la base del robot lista, el siguiente paso será colocar la batería en el repositorio que se encuentra en la parte posterior de la base principal e insertar cada una de las conexiones como se ubican en la Figura 17.

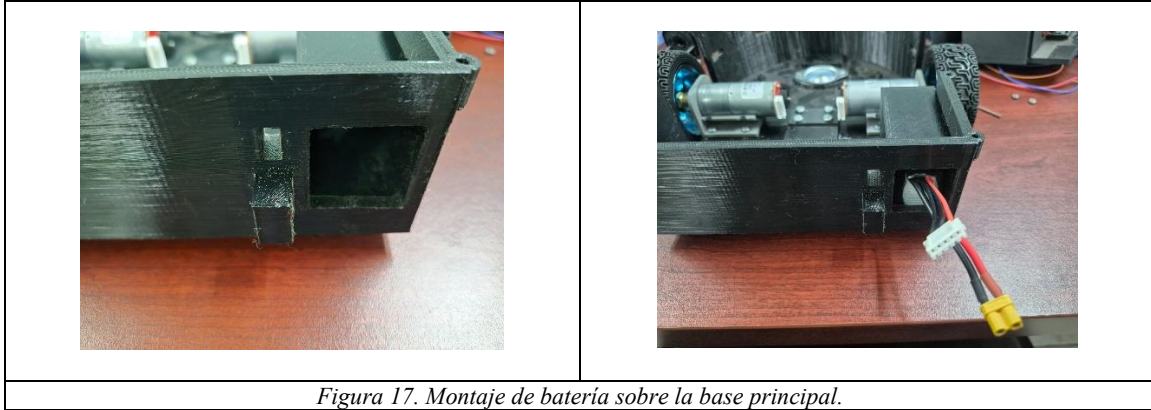


Figura 17. Montaje de batería sobre la base principal.

El montaje de los sensores se llevó en la parte frontal de la base principal, para ello se utilizaron 10 tornillos M3X5, en esta parte es importante colocar de buena manera el sensor, ya que pueden quedar mal orientados, para que esto no suceda se necesitan atornillar de a poco, los pines de cada sensor deben de colocarse dentro de la pequeña ranura que se encuentra entre los orificios. El montaje de los sensores se ilustra en la Figura 18.

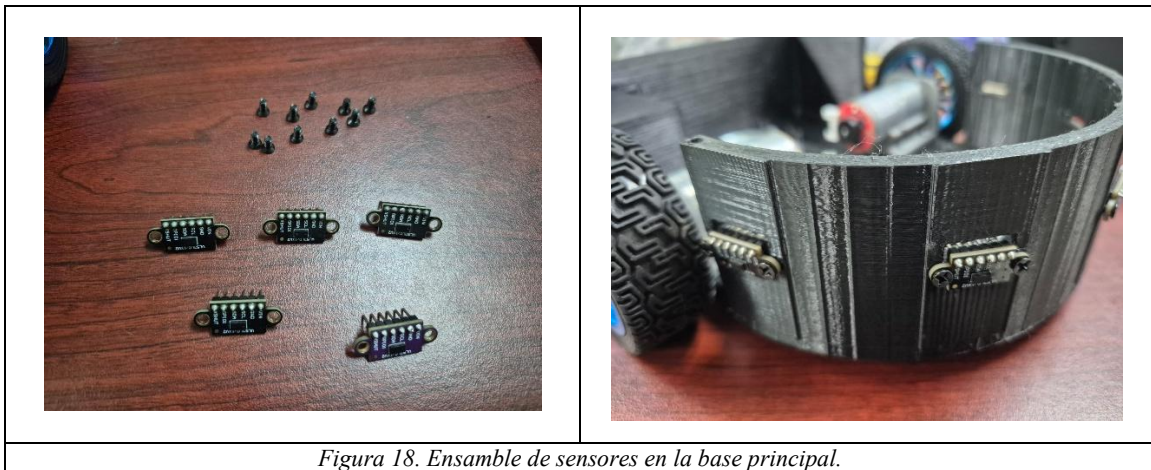
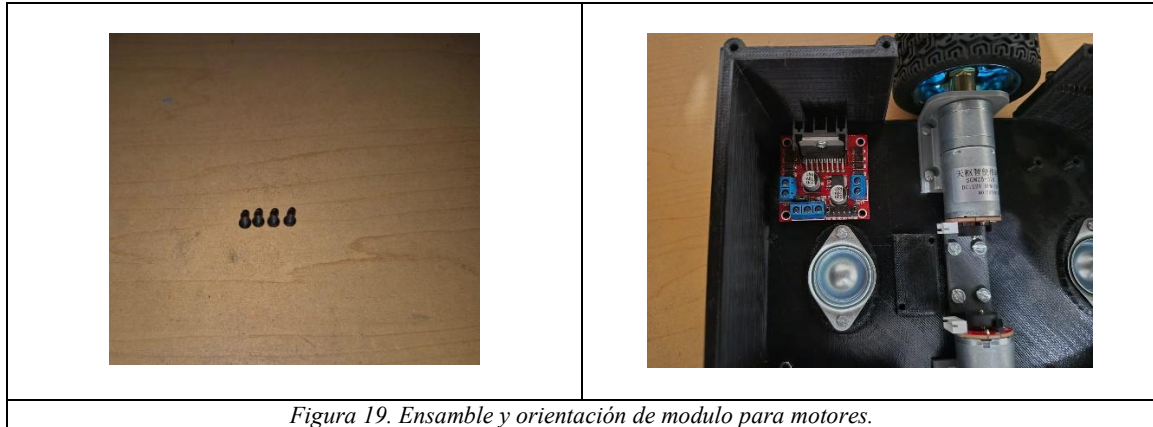
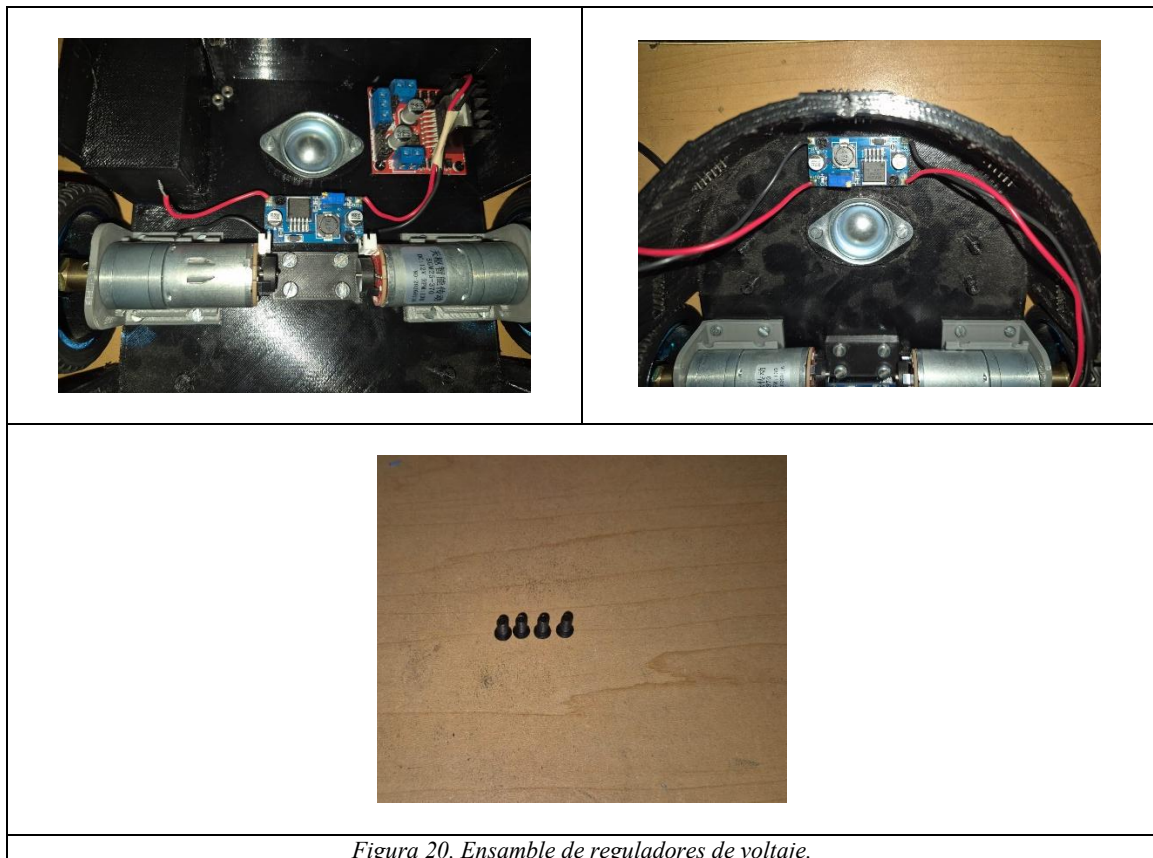


Figura 18. Ensamble de sensores en la base principal.

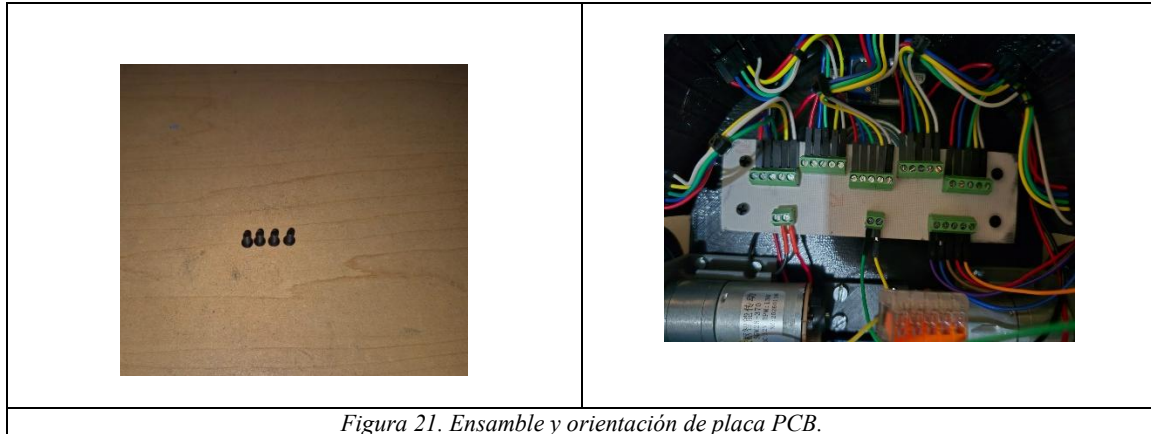
El módulo para controlar los motores se colocará en la parte lateral posterior de la base principal, para sujetarlo se ocuparon 4 tornillos M3X5, la orientación del módulo se muestra en la Figura 19, el propósito de la orientación es ordenar de una mejor manera los cables.



Para alimentar a cada componente, en la parte posterior de la base principal se colocó un regulador que entrega de salida 12 volts con los cuales se alimentaran los motores, en la parte frontal se colocó otro regulador configurado a 5 volts para alimentar los sensores. Para cada regulador se utilizaron 2 tornillos M3X5. En la Figura 20 se muestra la ubicación exacta de cada regulador.



Para finalizar con el ensamblaje de elementos en la parte interior de la base principal, se colocó la placa PCB en la parte frontal, se utilizaron 4 tornillos M3X5, los tornillos deben de ir en los 4 orificios que contiene la placa y estos deben de estar alineados con los 4 soportes que se encuentran en la parte frontal de la base, la orientación de la placa se observa en la Figura 21.



5. CONTROL Y COMUNICACIONES

5.1 Arquitectura de control

El desarrollo de proyectos relacionados con la robótica exige una distribución de tareas estrictamente basándose en la capacidad computacional de los elementos inmiscuidos, para garantizar respuestas eficientes en tiempo real y el procesamiento de algoritmos complejos de manera simultánea. Por tal motivo, y también para procurar un alcance mayor bajo la perspectiva descrita hacia el desarrollador, este prototipo opta por una arquitectura de control distribuida bajo un esquema similar al de Maestro-Eslavo.

Esta división de tareas delega el procesamiento de alto nivel (procesamiento de comandos remotos, gestión de *middleware* mediante ROS2) a un microprocesador, mientras que los procesos de bajo nivel (adquisición de señales, activación o interrupción de actuadores) son ejecutados por un microcontrolador. Por ende, la comunicación entre ambas capas de procesamiento se realiza de manera asíncrona a través de un bus de datos, proveniente de la conexión física hacia el puerto serial (USB), garantizando que el flujo de datos no comprometa la interacción de los elementos de hardware.



5.2 Raspberry Pi

La etapa de procesamiento de alto nivel es realizada por una Raspberry Pi 4, desempeñando el rol del dispositivo *Maestro* y la principal unidad de procesamiento del prototipo. Su elección se fundamenta en la capacidad que tiene para ejecutar de forma nativa un sistema operativo basado en Linux (específicamente Ubuntu) requisito fundamental para el entorno de *ROS2 Humble*.

Para implementar el *workspace* de *ROS2 Humble* (sección 7.3.1) es necesario instalar a la Raspberry Pi la versión del sistema operativo *Ubuntu 22.04*, la cual es una distribución de *Linux*. Posteriormente para una correcta gestión de archivos de código, se aconseja instalar adicionalmente en la placa, un visualizador de código fuente como *Visual Studio Code*, además del *IDE* de *Arduino* para separar la etapa de bajo procesamiento predicha, sin inmiscuir a todo lo concerniente con la implementación de *ROS2*.

Esto le garantiza al usuario una mayor disponibilidad al momento de probar y desarrollar algoritmos codificados para diversos microcontroladores, de forma independiente al resto del armado del robot diferencial, acoplándose al diseño final, sin depender físicamente del resto de la estructura física.

Finalmente, y como punto medular de esta sección, es necesaria la instalación de *ROS2 Humble*; para no caer en equivocaciones se aconseja seguir la metodología de instalación propuesta por la documentación oficial de ROS que puede consultar en el siguiente enlace:

<https://docs-ros-org.translate.goog/en/humble/Installation.html? x tr sl=en& x tr tl=es& x tr hl=es& x tr pto=tc>

NOTA: Todas las instrucciones referidas han sido orientadas hacia la Raspberry Pi 4, el usuario debe hacerse con los periféricos correspondientes para navegar dentro de la misma de ser necesario, es decir contar con al menos un teclado, mouse y monitor para desempeñar la codificación necesaria. Sin embargo, es posible también acceder por medio de comandos *SSH* o a través de un escritorio remoto, como se ejemplificará en la sección 7.3.3 correspondiente a la demostración del robot.

La instalación del software descrito no es del todo indispensable que el usuario la replique en su propia computadora, aunque puede serle útil para el prototipado, desarrollo, investigación o implementación de las múltiples funcionalidades de ROS.

5.3 Arduino

Desempeñando los algoritmos de bajo coste computacional, el *Arduino* seleccionado para el prototipo es una *Arduino Nano*, el cual posee características de procesamiento similares a otras placas comerciales, pero con un diseño más compacto (con un ancho y alto de 18 y 45 mm respectivamente), y un peso casi despreciable (de apenas unos 8 gramos). Con las condiciones precedentes, se abre paso a la modularidad del sistema apenas referida en el



apartado anterior, es decir, la capacidad de llevar a cabo el desarrollo de la lógica ejecutable por el microcontrolador de forma independiente.

6. HARDWARE

6.1 Alimentación

Una de las partes fundamentales en el desarrollo de todo proyecto relacionado con la robótica, es el suministro de energía; siendo los problemas más recurrentes una demanda que sobrepase los límites que un componente puede entregar, o el suministro de la cantidad imprecisa (por debajo o por encima de la requerida) para alimentar un circuito.

Partiendo de que el prototipo en sí mismo debe prescindir de todo cableado externo para su alimentación y que como en la mayoría de los proyectos, es necesario el uso de baterías, lo siguiente es bajar los niveles de voltajes y corrientes a las distintas necesidades que existen. Evidentemente, de todos los componentes aquellos que requieren mayor energía son los motores *GM 25-370*, sin embargo, es necesario regular su alimentación a pesar de utilizar el *driver L298N* como se explicará en la sección posterior.

La siguiente tabla resume los niveles de suministro en cada uno de los componentes, así como el medio por el cual debe de ser regulada dicha alimentación

Tabla 1. Características de componentes del robot.

SUMINISTRO PRINCIPAL		
Batería LiPo Ovonix	850 mAh 14.8 V 12.58 Wh	
COMPONENTES	ALIMENTACIÓN	MÓDULO
Raspberry Pi 4	$V \geq 5V$ $I = 1.8A - 3A$	Mini 560
Arduino NANO	$V = 4.8V - 5.2V$ $I = 0.5A - 0.9A$	Raspberry Pi 4
Motores GM 25-370	$V = 6V - 15V$ $V_{nominal} = 12V$ $I = \text{segun la carga (0.1A - 3A)}$	LM2596S y Driver L298N
Encoders	$V = 3.3V - 5V$	Arduino Nano

	$I = 10mA$	
Sensores VL53L0X	2.6 – 3.5 V	LM2596S

De la tabla anterior se rescatan ciertos puntos relevantes, uno de ellos es que algunos de los componentes del hardware reciben su alimentación directamente de los microcontroladores (como en el caso de los encoders o el *Arduino NANO*) por lo que no debe existir preocupación alguna en esos casos debido a la regulación de voltaje o una demanda excesiva pues son dispositivos estables que no incurren en demandas atípicas de voltaje o de corriente en la operación.

Sin embargo, aquellos que utilizan módulos de regulación externos ameritan una justificación en su implementación; siendo principalmente la seguridad frente a sobrecarga o consumo excesivo de potencia. Durante la operación del prototipo, los motores requieren inicialmente un pico de corriente para romper la inercia de su eje, lo que ocasionaría un shutdown del sistema si no se contempla en el diseño; por otra parte, toda placa Raspberry Pi es bastante susceptible a la alimentación que se le otorga por lo cual es mejor procurar que esta permanezca estable frente a cualquier circunstancia.

La solución más asequible a ambos problemas es el uso de los módulos LM2596S y Mini560, cuyas especificaciones técnicas más relevantes aparecen a continuación:

Tabla 2. Características de reguladores de alimentación.

	LM2596S	Mini560
Topología	Buck ($V_{salida} < V_{entrada}$)	
$V_{in}(Máx)$	40V	7V
$V_{in}(Mín)$	4.5V	20V
V_{out}	3.3V – 37V	5V
$I_{out}(Máx)$	3A	5A
Temperatura de operación	$-40^{\circ}C$ a $125^{\circ}C$	$-40^{\circ}C$ a $85^{\circ}C$
Protección contra sobre corriente	Si	Si



Figura 22. Regulador LM2596.



Figura 23. Regulador Mini 560.

6.2 Actuadores

Asociados muy a menudo con las aplicaciones finales del robot, estos dispositivos llevan a cabo las acciones programadas en la etapa de control. Puesto que en este prototipo solo se cuenta con la movilidad y el sensado de distancia, el cual ha sido considerado en otra sección particular, podemos reducir la presencia de actuadores a los motores del robot.

Estos motores (Motores *GM 25-370*) cuentan con encoders de cuadratura rotativos incrementales, que funcionan bajo el principio del efecto hall, es decir, mediante la detección magnética en el paso, lo que les permite trabajar en entornos extremos (altas temperaturas, salpicaduras, presencia de polvo, etc.) sin necesidad de mantenimiento. Sin embargo la característica más relevante es que cuenta con dos canales de detección desfasados entre si 90° ; dichos canales generan pulsos digitales gracias a un disco magnético montado en el eje trasero del motor, por lo cual no solo es posible realizar una equivalencia de movimiento en el mundo real en base a las vueltas del motor y su relación de engranes en la caja reductora, sino que además es posible conocer la dirección de giro según el orden de detección registrado (los detalles en la lógica pueden consultarse en una hoja de datos o en el código de Arduino descrito en la sección 7.2.1).

Las siguientes características son de interés para el usuario:

Tabla 3. Características de motores.

Voltaje Nominal	12V
Velocidad sin Carga	140 RPM a 0.1 A
Máxima eficiencia	Carga 1.7Kg · cm/110rpm/2.1w/0.33A
Máxima potencia	Carga 4.3Kg · cm/70rpm/3.2w/0.75A
Bloqueo del eje (<i>Stall</i>)	Stall torque 8.5Kg · cm/Stall current/1.4A
Relación de reducción	1: 45
Holzer (resolución del encoder)	11
Pulsos por revolución	Holzer · R. de reduccion = $11 \cdot 45 = 495$

Por último, el llevar a buen término un control extensivo del movimiento de los motores no sería posible sin el driver L298N, el cual está basado en el circuito integrado del *punte*

H dual L298 que permite mantener líneas de suministro separadas a ambos motores, inversión de giro, y variación de velocidad por el método de modulación de ancho de pulso *PWM* otorgado por el microcontrolador (el *Arduino NANO* en estas circunstancias). Algunas de sus características más relevantes consideradas en el diseño aparecen en la siguiente tabla:

Tabla 4. Características de driver *L298N*.

Voltaje de entrada	3.2V a 40V
Voltaje de suministro	5V a 35V
Corriente Máxima	2A
Corriente durante operación	0A – 36mA
Máximo consumo de potencia (a 75°C)	20W
Temperatura de operación	–25°C a 130°C

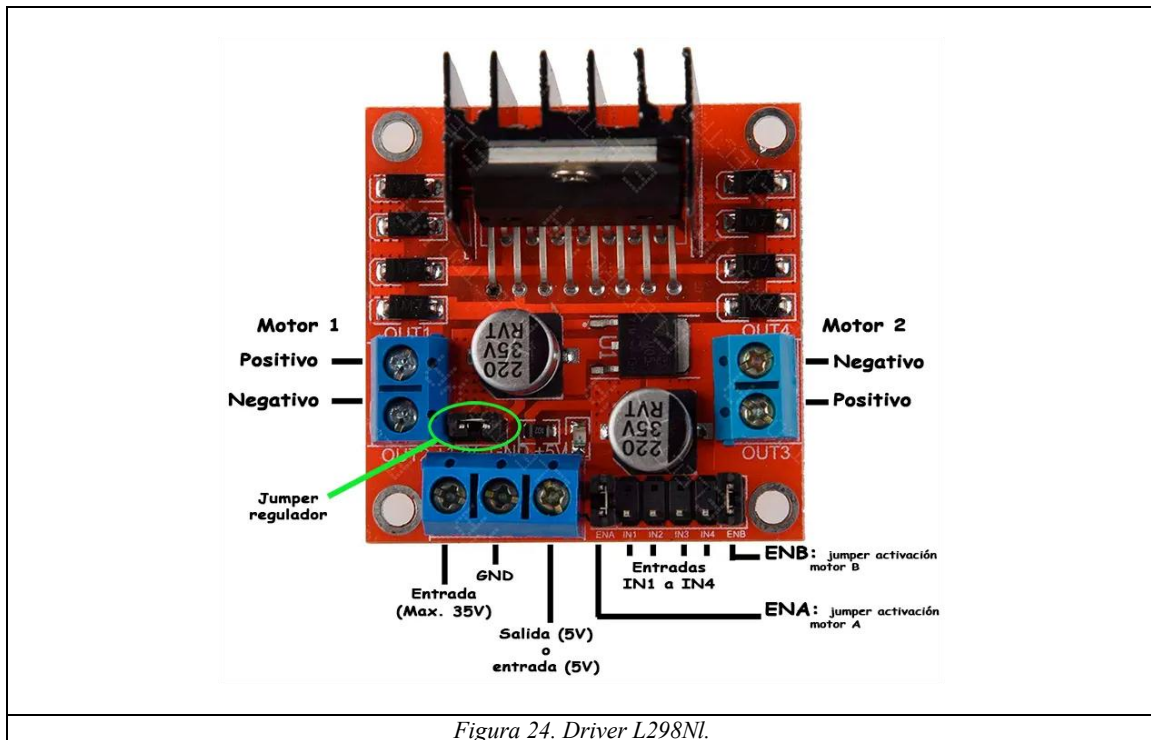


Figura 24. Driver *L298N*.

6.3 Placa PCB

Como se mencionó previamente, el robot cuenta con cinco sensores de distancia VL53L0X montados en la parte frontal, tal como se observa en la Figura 25.

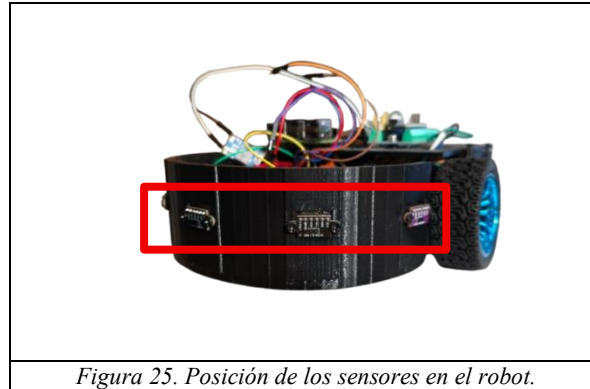


Figura 25. Posición de los sensores en el robot.

Estos sensores utilizan el protocolo de comunicación I2C para transmitir las mediciones de distancia. Cada sensor dispone de los siguientes pines: comunicación (SDA y SCL), alimentación (VCC y GND), habilitación (XSHUT) y un pin de propósito general (GPIO1), como se muestra en la Figura 26.

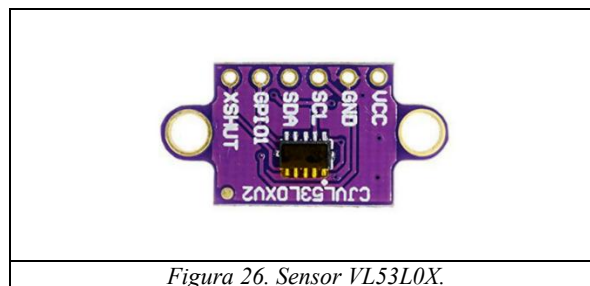


Figura 26. Sensor VL53L0X.

Para la implementación en el robot, se emplean cinco de los seis pines disponibles por sensor. Considerando los cinco sensores, esto implicaría el uso de un total de 20 cables para su conexión individual. Con el objetivo de simplificar el cableado y mejorar la organización interna, se diseñó una placa de circuito impreso (PCB) que reduce el número de conexiones a únicamente nueve cables: cinco para el control individual de cada sensor, dos para la comunicación I2C y dos para la alimentación.

La PCB tiene dimensiones de 121×36 mm y se encuentra instalada en el compartimento frontal del robot, donde está fijada mediante cuatro tornillos, como se muestra en la Figura 27.

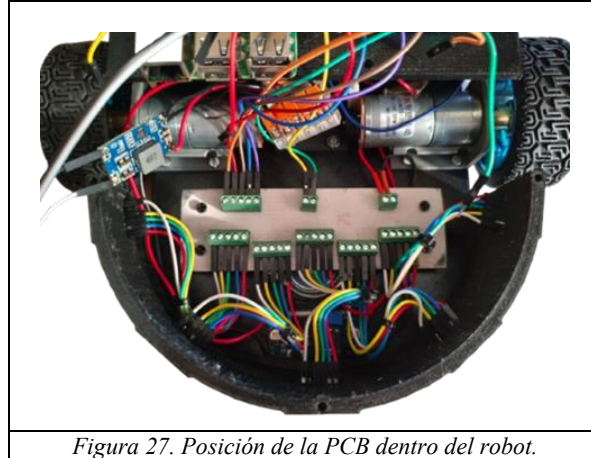


Figura 27. Posición de la PCB dentro del robot.

El diagrama de la PCB se presenta en la Figura 28. El diseño fue realizado utilizando el software de código abierto KiCad.

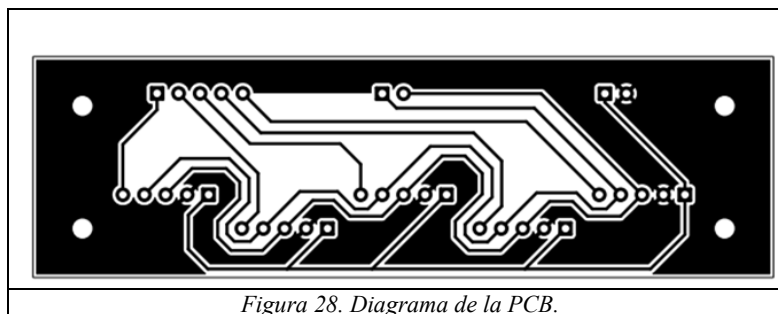
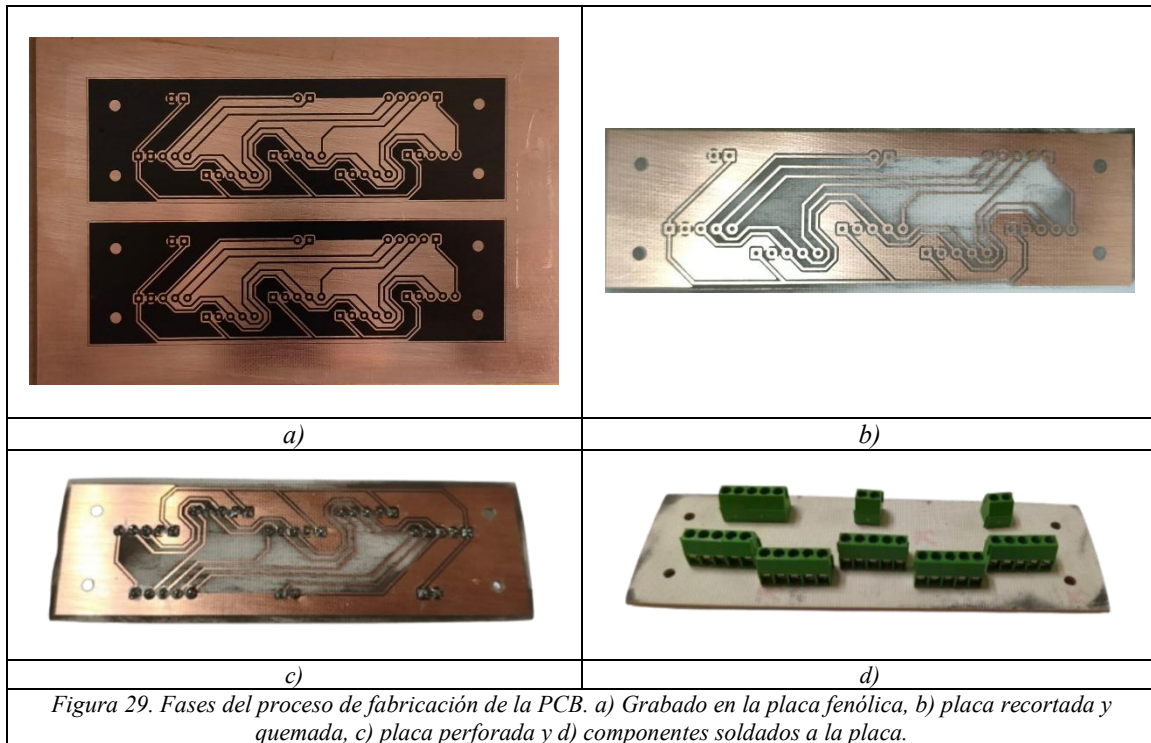


Figura 28. Diagrama de la PCB.

El proceso de fabricación de la PCB fue el siguiente:

1. Se imprimió el diseño del circuito sobre una placa fenólica.
2. Se recortó la placa a las dimensiones requeridas.
3. Se sumergió la placa en una solución de cloruro férrico y agua en proporción 2:1 para eliminar el cobre que no esté cubierto por la tinta.
4. Una vez completado el grabado, la placa se retiró y se lavó con abundante agua.
5. Finalmente, se realizaron las perforaciones necesarias y se soldaron los componentes.

Este proceso se ilustra en las siguientes imágenes:

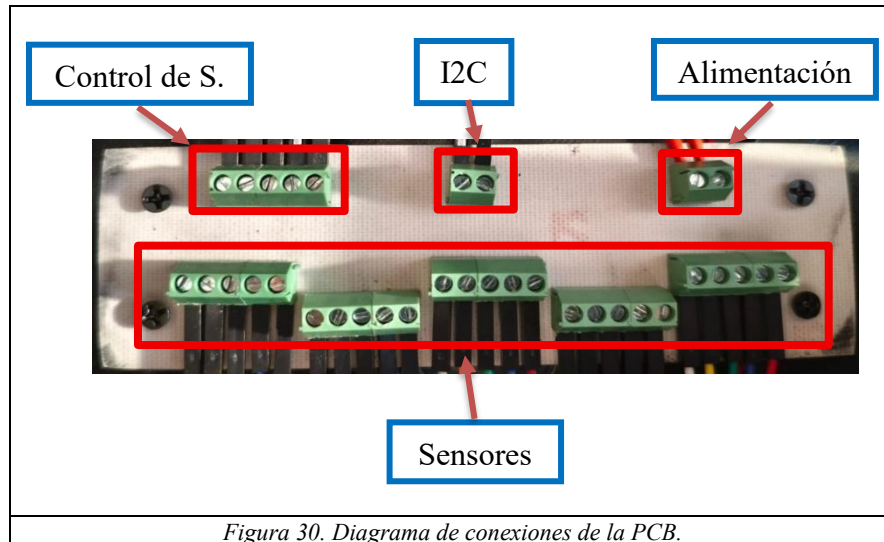


Los componentes utilizados en la PCB son los siguientes:

- 7 bloques de terminales de 3.5 mm con 2 pines
- 6 bloques de terminales de 3.5 mm con 3 pines

La ventaja de los bloques de terminales es que se pueden unir para formar bloques con más pines.

El diagrama de conexiones de la placa se muestra en la Figura 30.



Las conexiones están de la siguiente manera y se mencionan de izquierda a derecha:

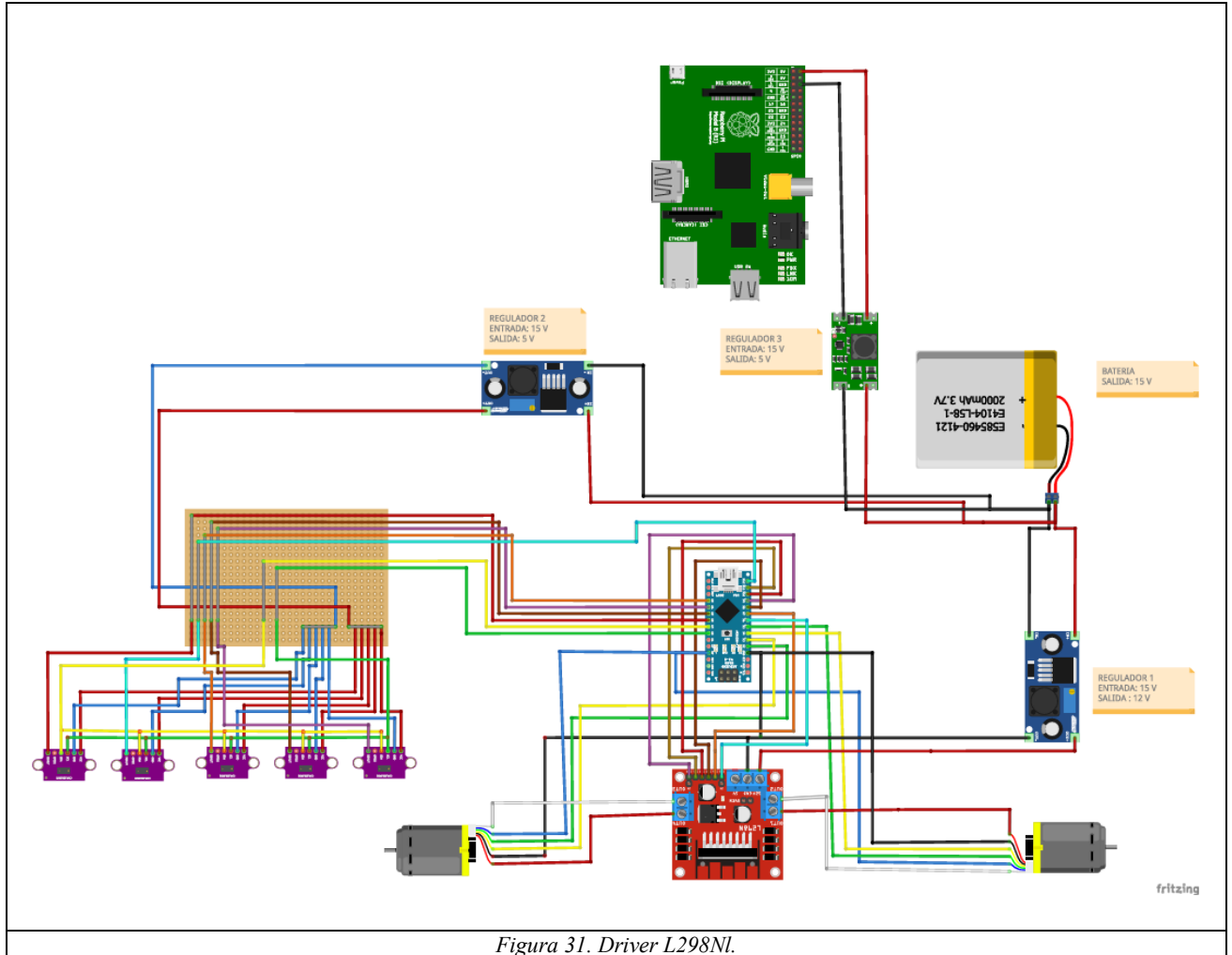
- Sensores: XSHUT, SDA, SCL, GND y VCC. Los sensores están enumerados de izquierda a derecha.
- Control de los Sensores: XSHUT1, XSHUT2, ..., XSHUT5.
- Comunicación I2C: SDA y SCL.
- Alimentación: VCC y GND.

Los archivos utilizados para la placa PCB se pueden consultar en el siguiente enlace:

https://github.com/Yahir-Baeza-17/RoboKit_sensors.git

6.4 Flujo de conexiones

En la Figura 31 se observan las conexiones finales de todos los componentes con los que cuenta el robot, los reguladores de voltaje cuentan con etiquetas para ubicar a que componentes están alimentando. En el prototipo físico del robot, cada una de las conexiones esta etiquetada para saber a qué entrada o salida del dispositivo deben de estar conectadas.



Para obtener una mejor visualización del archivo de conexiones, es necesario dirigirse al siguiente enlace donde se encuentra el archivo de conexiones en formato PDF:

<https://drive.google.com/file/d/1ncNqyscPOZh0cvkc9B3dF30TkAiiUyPe/view?usp=sharing>

7. SOFTWARE

Todos los archivos de código para el control de motores están alojados en el siguiente repositorio de GitHub:



<https://github.com/JhonLemon/Robokit-Laviria/tree/main>

Para tener una comprensión más profunda del funcionamiento de los códigos se aconseja rotundamente consultar los códigos comentados dentro del repositorio mencionado.

7.1 Integración e interacción

Desde la perspectiva del software, la sincronización entre la lógica de alto y bajo nivel se logra estableciendo un canal de comunicación bidireccional donde la Raspberry Pi dicta las velocidades a ambos motores a modo de *setpoints*, y el Arduino recibe, procesa y traduce estas instrucciones en señales eléctricas para los motores, en una operación de lazo cerrado completada por la lectura constante de pulsos proveniente de los encoders para realizar un control *PID*.

El flujo de trabajo propuesto requiere que el usuario clone el repositorio oficial del proyecto dentro de un espacio de trabajo de ROS2, dentro de la Raspberry Pi, asimismo el firmware de control de los motores debe ser cargado a la placa de Arduino mediante su propio IDE.

7.2 Arduino

El microcontrolador fue seleccionado para actuar como el núcleo de operaciones en tiempo real (consulte sección 5), haciendo un uso intensivo de las interrupciones por hardware provenientes de los encoders de cada motor. Tal es la razón de que se hayan seleccionado los pines 2 y 3 de la placa Arduino NANO para uno de los dos canales disponibles en cada motor, pues son los únicos pines digitales capaces de detectar interrupciones externas directas producidas por los flancos de subida y de bajada en las señales.

Como aspecto a recalcar esta la tasa de baudios a la cual se establece el flujo de información entre los dispositivos, la cual se debe fijar a 57 600 baudios (aprox. 5760 bytes por segundo) para mantener un control estable a lo largo de la compilación.

7.2.1 Motores

El flujo de trabajo dentro código *SS_CONTROL_MOTORES.ino* se encuentra realizando cíclicamente dos tareas, la primera de ellas es la lectura y gestión de comandos del puerto serial y la segunda es la del control *PID*.

Dejando de lado la declaración de variables, definición de constantes, y demás pormenores necesarios para la compilación; la primer tarea, correspondiente a la lectura de comandos se encarga procesar la información recibida por el puerto serial para reinterpretarla según los comandos establecidos en el código (los cuales el usuario puede leer en los comentarios del mismo o en su despliegue durante la ejecución del programa a través de la ventana serial, ya sea en el Arduino IDE o a través de su respectivo nodo de ROS), condicionando



las decisiones que el microcontrolador debe tomar para que la etapa final de potencia conformada por los motores, refleje las instrucciones precisas a través de todo el hardware descrito en secciones previas (consultar secciones 6.2 y 6.4).

La segunda tarea es la aplicación de un control PID discreto por medio del método de diferencias finitas, el cual está establecido para operar con una frecuencia justa y por medio de variables del proceso con valores definidos (kp , ki , kd) para el mayor aprovechamiento y comportamiento frente al establecimiento de nuevos objetivos (*setpoints*).

Ambos procesos, ejecutándose de forma simultánea con la detección de los canales por medio de las rutinas de servicio de interrupción *ISR* (atendidas mediante los pines 2 y 3 como se describió en la sección anterior), concluyen en una función encargada de procesar la instrucción objetivo actual y realizar el control PWM a través de instrucciones en los pines digitales, las cuales son entregadas al driver L298N para ser finalmente traducidas en el movimiento requerido de ambas ruedas dentro del robot.

7.3 ROS

Fundamentalmente, la implementación de ROS 2 Humble dota al prototipo de una infraestructura de comunicaciones estandarizada basada en DDS (*Data DistributionService*), haciendo hincapié de nuevo en la escalabilidad del modelo como una plataforma de desarrollo lista para integrar algoritmos de navegación autónoma (*Nav2 – Navigation 2*), SLAM (*Simultaneous Localization and Mapping*) y de visión por computadora.

7.3.1 Estructura del espacio de trabajo (*Workspace*)

En ROS 2, todo proyecto debe residir dentro de un Espacio de Trabajo o *Workspace*. El usuario debe replicar esta estructura bajo sus propósitos para alojar los paquetes desarrollados, por ende, se plantea la siguiente metodología para ser implementada en la terminal de Ubuntu:

- 1- Creación de un Espacio de Trabajo: Dentro de la terminal de Ubuntu, se crea un directorio, los directorios padre necesarios (en caso de que no existan) para la ruta y una carpeta *source* (*src*) que es donde reside todo el código fuente.

```
mkdir -p ~/robot_ws/src
cd ~/robot_ws/src
```

- 2- Clonación del repositorio: Para descargar el paquete (es decir el conjunto de archivos involucrados con los nodos y su compilación, incluyéndolos a estos) desde el repositorio.

```
git clone https://github.com/JhonLemon/Robokit-Laviria.git
```



Por último, al momento de realizar una compilación total del espacio de trabajo, es necesario seguir estos pasos desde la terminal:

- 1- Ubicación: Primero debemos alojarnos en la raíz del workspace para compilar el entorno utilizando la herramienta **colcon**, la cual enlaza las librerías de *Python* y *ROS2* con los scripts.

```
cd ~/robot_ws
colcon build
```

- 2- Sourcing: Para que el sistema operativo reconozca los nuevos nodos ejecutables, se debe realizar un proceso conocido como sourcing, que no es más que cargar las configuraciones del entorno para ejecutarlos dentro de la terminal, esto por medio de la siguiente instrucción:

```
source install/setup.bash
```

- 3- Compilación: Para finalizar con el proceso y realizar la compilación es necesario dar la instrucción a ROS2 de que ejecute el nodo específico mediante la siguiente instrucción:

```
ros2 run nombre_del_paquete nombre_del_nodo
```

NOTAS: Los nombres asignados en el proyecto para el nombre del paquete es `arduino_bridge` y para los nodos son `serial_node` y `demo_node`; además en caso de ser necesario, el usuario puede tener múltiples nodos ejecutándose a la vez en distintas ventanas dentro de la terminal de Ubuntu.

7.3.2 Nodo de comunicación serial

El script que se encuentra dentro del archivo `serial_node.py` constituye el puente principal entre el entorno de ROS2 y el control realizado con el Arduino, pues inicializa una comunicación serial a una tasa de 57600 baudios a través del puerto físico `dev/ttyUSB0`.

Su característica más destacable es el uso de la técnica de multihilo; pues en el desarrollo de software para aplicaciones dentro del mundo de la robótica, las funciones que están a la espera de entradas del usuario retienen intencionalmente la ejecución del programa. Es así como el aprovechamiento de esta técnica evita que las rutinas internas de ROS2 queden pausadas; siendo que dentro del nodo se ejecuta un hilo secundario dedicado exclusivamente a esta tarea, mientras que el hilo principal (ejecutado por `rclpy.spin()`) continúe gestionando los tópicos, recibiendo mensajes de otros nodos y leyendo las respuestas asíncronas provenientes del microcontrolador durante la ejecución en tiempo real.



7.3.3 Nodo de demostración

Alojado en el archivo `demo_node.py`, este código mejora las posibilidades del nodo anterior mediante una interfaz gráfica de usuario *GUI*, construida con las herramientas de la librería `tkinter`. A diferencia del nodo serial que espera comandos a través de tópicos de ROS, el nodo de la demostración se enlaza directamente hacia el puerto serial de Arduino para recrear movimientos básicos.

Dentro de la interfaz se encuentra la réplica de una botonera direccional con 4 opciones básicas de movimiento (avance, retroceso y giros en sentido horario y antihorario) con la capacidad de regular la velocidad del movimiento a través de un *slider* el cual reemplaza las instrucciones numéricas de velocidad y otorga conjuntamente con la botonera, los comandos necesarios para operar dentro de un rango de velocidades (de 0 a 120 rpm) en todas las combinaciones de tracción posibles por ambas ruedas. Al igual que el nodo serial, la interfaz hace uso de la técnica de multihilo para separar el bucle infinito de la ventana (a través de `app.mainloop()`) de los procesos de ROS2 ejecutándose en segundo plano.

8. CONEXIÓN A LA RASPBERRY

En esta sección se describe el procedimiento necesario para establecer una conexión remota entre la Raspberry Pi y un equipo de cómputo.

Importante: Para que la conexión funcione correctamente, ambos dispositivos deben estar conectados a la misma red Wi-Fi.

8.1 Configuración de la Raspberry Pi

Para que la Raspberry Pi admita la conexión remota se necesita seguir el siguiente procedimiento:

Acceder a la terminal desde la Raspberry Pi y teclear los siguientes comandos para actualizar el sistema e instalar el servicio de escritorio remoto:

```
sudo apt update
sudo apt install xrdp
```

Finalizada la instalación, se debe de habilitar el servicio y permitir el puerto de conexión:

```
sudo systemctl enable xrdp
sudo ufw allow 3389/tcp
```



Posteriormente hay que hacer la configuración de permisos adicionales, para ello necesitamos abrir el archivo de configuración usando el siguiente comando:

```
sudo nano /etc/polkit-1/localauthority/50-local.d/45-allow-colord.pkla
```

Y en el archivo, copie y pegue el siguiente contenido:

```
[Allow Colord all Users]
Identity=unix-user:*
Action=org.freedesktop.color-manager.create-
device;org.freedesktop.color-manager.create-
profile;org.freedesktop.color-manager.delete-
device;org.freedesktop.color-manager.delete-
profile;org.freedesktop.color-manager.modify-
device;org.freedesktop.color-manager.modify-profile
ResultAny=no
ResultInactive=no
ResultActive=yes
```

Guarde los cambios presionando Ctrl + O, luego Enter, y salga con Ctrl + X.

Con la configuración anterior ya es posible conectarse remotamente a la Raspberry Pi, pero hay un problema; para establecer el enlace no necesita haber ninguna sesión iniciada, por defecto la Raspberry Pi inicia sesión automáticamente, por lo que es necesario deshabilitar esta opción, para ello se realiza lo siguiente:

En la terminal, ejecute:

```
sudo nano /etc/gdm3/custom.conf
```

Y en el archivo que se abre busque las líneas:

```
AutomaticLoginEnable=true
AutomaticLogin=[usuario]
```

Modifíquelas de la siguiente forma:

```
#AutomaticLoginEnable=false
#AutomaticLogin=[usuario]
```

Guarde los cambios presionando Ctrl + O, luego Enter, y salga con Ctrl + X.

Una vez completados estos pasos, la Raspberry Pi estará configurada para aceptar conexiones remotas al encenderse.



8.2 Configuración de Windows

Para conectarse desde un equipo con Windows, hay que hacer algunas modificaciones para que funcione adecuadamente, para esto siga el siguiente procedimiento:

Primero hay que habilitar la plataforma de máquina virtual, para esto presione la tecla Windows y busque “Activar o desactivar las características de Windows” y presione Enter.

En la lista de opciones, active Plataforma de máquina virtual y después haga click en aceptar.

Ahora, hay que abrir la conexión a escritorio remoto. Para esto busque la aplicación “Conexión a escritorio remoto” y ejecútela. En el campo correspondiente, introduzca la dirección de la Raspberry Pi y llene las credenciales que le pidan.

8.3 Obtención la dirección IP de la Raspberry Pi

Para obtener la dirección IP de la Raspberry Pi en la red, abra la terminal y ejecute el siguiente comando:

```
hostname -I
```

Una vez obtenida la dirección IP, introdúzcala en la aplicación de escritorio remoto en Windows para establecer la conexión.

9. RANGE SENSING

9.1 Sensores

El sensor implementado para obtener las mediciones de distancia fue el VL53L0X, este sensor cuenta con una alta precisión basado en la tecnología Time of Flight (Tiempo de vuelo), es fabricado por STMicroelectronics y cuenta con un láser VCSEL de 940 *nm*, una de las razones por las cuales fue elegido fue su pequeño tamaño de aproximadamente $4.4 \times 2.4 \times 1.0 \text{ mm}$ lo cual es ideal para el tamaño del chasis del robot, además tiene un alcance de medición absoluto de 2 *m*. Algunas de las ventajas con las que cuenta el sensor son las siguientes: el alcance informado es independiente de la reflectancia del objeto, funciona a altos niveles de luz ambiental infrarroja, compensación avanzada de diafonía óptica integrada para simplificar la selección de vidrio de cubierta. En la Figura 32 se observa el sensor VL53L0X y sus respectivos pines de conexión.



Figura 32- Sensor VL530X.

En sensor cuenta con los dos respectivos pines de alimentación (V_{in} y GND), además de los 2 pines de datos SCL (señal de reloj serial) y SDA (señal de datos serial), también cuenta con un pin de interrupción ($GPIO1$), finalmente se encuentra el pin de reset o control ($XSHUT$). La Tabla 5 muestra las especificaciones técnicas del sensor.

Tabla 5. Especificaciones técnicas del sensor VL53L0X.

Característica	Detalle
Empaquetado	LGA 12 óptico
Tamaño	$4.40 \times 2.40 \times 1.00 \text{ mm}$
Voltaje de operación	$2.6 - 3.5 \text{ V}$
Temperatura de operación	$-20 - 70 \text{ }^{\circ}\text{C}$
Emisor infrarrojo	940 nm
I^2C	Hasta 400 KHz para bus serial (modo rápido) Dirección: $0x52$

9.2 Disposición de sensores

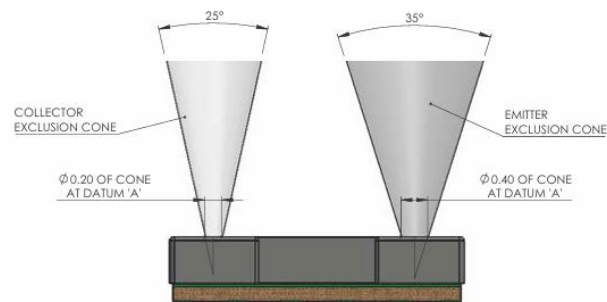


Figura 33- Campo de visión (Fov) del emisor y receptor.

La distribución de los sensores se basa en el campo de visión (*Fov*) del emisor y receptor, ya que se buscó una separación más grande que el campo de visión mayor que en este caso es el del emisor de 35° , por lo tanto, se propuso una separación de 40.5° entre cada sensor partiendo de la parte frontal del sensor como se observa en la Figura 34. El objetivo de esta distribución fue no mezclar los haces de luz infrarroja entre cada uno de los sensores, además de buscar tener una mayor cantidad de datos sobre el entorno en el que se encuentra el robot, esto da hincapié en tener la posibilidad de mapear el entorno para aplicaciones futuras.

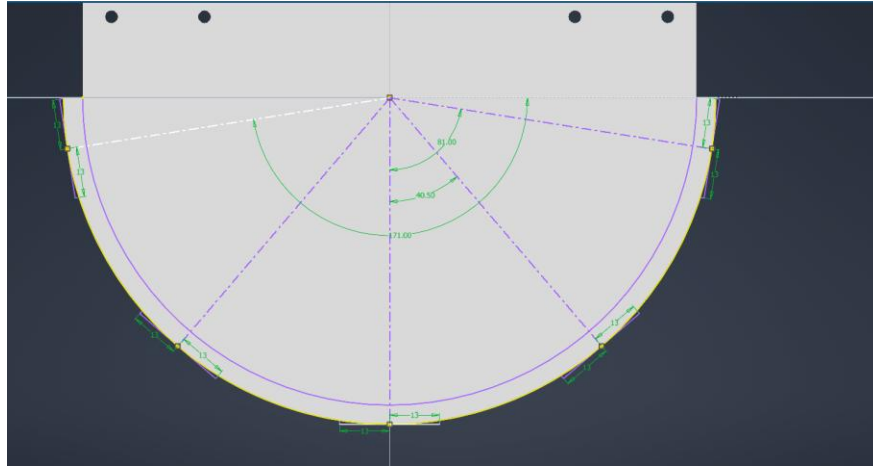


Figura 34- Distribución de los sensores en la sección frontal del chasis.

La elevación de cada sensor fue de 27.5 mm sobre el espesor de 5 mm de la base con la finalidad de que el robot pueda detectar la mayor cantidad de obstáculos presentes en su entorno, además la altura aproximada del robot es 15 mm sobre el nivel del piso, es evidente que algunos obstáculos no se detecten porque su altura es inferior al rango de visión de los sensores y esto genera un área de mejora en los futuros diseños del chasis del robot. En la Figura 35 ilustra un ensamble de la sección frontal del chasis con los sensores.

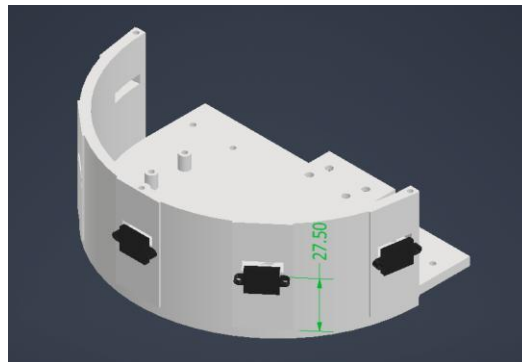


Figura 35- Ensamble de sección frontal del chasis con sensores.



9.3 Codificación Arduino

El código implementado para controlar cada uno de los sensores se realizó por medio de la librería `#include <VL53L0X.h>`, la cual tiene funciones especializadas en el sensor VL530X como: inicializar el sensor, cambiar su dirección de memoria y medir la distancia en milímetros. La comunicación requerida es por medio del protocolo I^2C , para ello se incorporó la librería `#include <Wire.h>`. En el apéndice A se muestra el código completo utilizado para la lectura de los sensores.

9.4 Espacio de trabajo (Workspace) para lectura de sensores.

Se creó un espacio de trabajo a parte para programar la comunicación de los sensores con el Arduino por medio de un nodo de ROS, el espacio de trabajo se programó desde una computadora y no desde la raspberry, por lo tanto, este apartado queda en desarrollo, su culminación sería unir el código de Arduino de los sensores al código principal, además de integrar el nodo de comunicación de los sensores, al espacio de trabajo principal donde se encuentra el nodo de comunicación de los motores. En los apartados anteriores ya se mencionó como instalar ROS, así que en esta sección simplemente se explicará como se creó el espacio de trabajo en Ubuntu.

1- Creación de un Espacio de Trabajo: Dentro de la terminal de Ubuntu, se crea una carpeta donde se guardarán los archivos principales, es importante mencionar que dentro de un espacio de trabajo se debe de tener la carpeta `source` (`scr`), dentro de esta carpeta se encuentra el código principal programado en python.

```
Mkdir -p ~/robot_ws/scr
```

Para acceder a la carpeta `source` se debe de aplicar el siguiente comando:

```
cd ~/robot_ws/scr
```

2- Clonación del repositorio: Para descargar el espacio de trabajo donde se encuentra el nodo de los sensores y todos los demás archivos involucrados se aplica el siguiente comando:



```
git clone https://github.com/Yahir-Baeza-17/RoboKit_sensors.git
```

9.4.1 Compatibilidad de puertos USB

Al estar manipulando el Ubuntu en la computadora, algunas veces se llegan a presentar problemas con la compatibilidad en los puertos USB, para ello es importante seguir los siguientes pasos para no presentar problemas de compatibilidad con los puertos.

Se ejecuta el Power Shell como administrador y verificaremos que puerto es el que ocupa el Arduino al conectarlo a la computadora, para ello se debe de conectar el Arduino a la computadora, se escribirá el siguiente comando que nos mostrará la lista de puertos USB que están conectados a la computadora:

```
usbipd list
```

Para saber en qué puerto está el Arduino, se debe de aparecer el nombre del chip que tiene el Arduino, por ejemplo:

```
USB-SERIAL CH340 (COM10)
```

En la parte izquierda se ubicarán números como 1-1, 2-2, 1-2, etc. Estos números son los que ocuparemos, ahora se aplicará el siguiente comando que permite compartir el puerto donde se encuentra el Arduino con el entorno de Ubuntu:

```
usbipd bind --busid 2-2
```

Es importante mencionar que el numero 2-2 puede cambiar dependiendo del puerto. El siguiente comando permite agregar el puerto de manera automática al entorno de Ubuntu cada que lo detecta la computadora, si la computadora se apaga o reinicia, solo este comando se tiene que escribir de nuevo en el Power Shell.

```
usbipd attach --wsl --busid 2-2 --auto-attach
```



9.4.2 Compilación del nodo de lectura de los sensores

1- Ubicación: Como primer paso se debe acceder al espacio de trabajo por el comando `cd ~/robot_ws`, después se necesita compilar el espacio de trabajo por medio del comando `colcon build`.

2- Sourcing: Este proceso se encarga de cargar las configuraciones del entorno para ejecutarlos dentro de la terminal, para ello se aplica el siguiente comando:

```
source install/setup.bash
```

3- Permisivos: Se necesita dar permisos al entorno de Ubuntu para utilizar el puerto serial por medio del siguiente comando:

```
sudo chmod 666 /dev/ttyUSB0
```

Nota: Es importante verificar el nombre del puerto USB al que está conectado el Arduino, si el puerto es incorrecto el entorno de Ubuntu no permitirá utilizar el puerto.

4- Ejecución: Finalmente, se ejecutará el nodo, pero antes es importante mencionar que el paquete donde se encuentra el nodo es `sensor_arduino` y el nombre del nodo es `sensor_reader`.

```
ros2 run sensor_arduino sensor_reader
```

9.4.3 Nodo de lectura de los sensores

El script principal se encuentra en el archivo `sensor_reader.py`, el funcionamiento consiste en una comunicación de 57600 baudios por medio del puerto `/dev/ttyUSB0`, el nodo contiene un tópico de tipo publicador, para que nodos externos se puedan subscribir al tópico y puedan visualizar el valor de las distancias. Para el procesamiento de la cadena de datos enviada por el Arduino, el nodo particiona la cadena y la guarda en un multi arreglo numérico para publicarlo en la terminal de la computadora. Si se quiere conocer el funcionamiento más a profundidad del código, es recomendable leer el código, ya que este cuenta con comentarios específicos en cada una de sus líneas y se encuentra en el siguiente enlace:

https://github.com/Yahir-Baeza-17/RoboKit_sensors.git